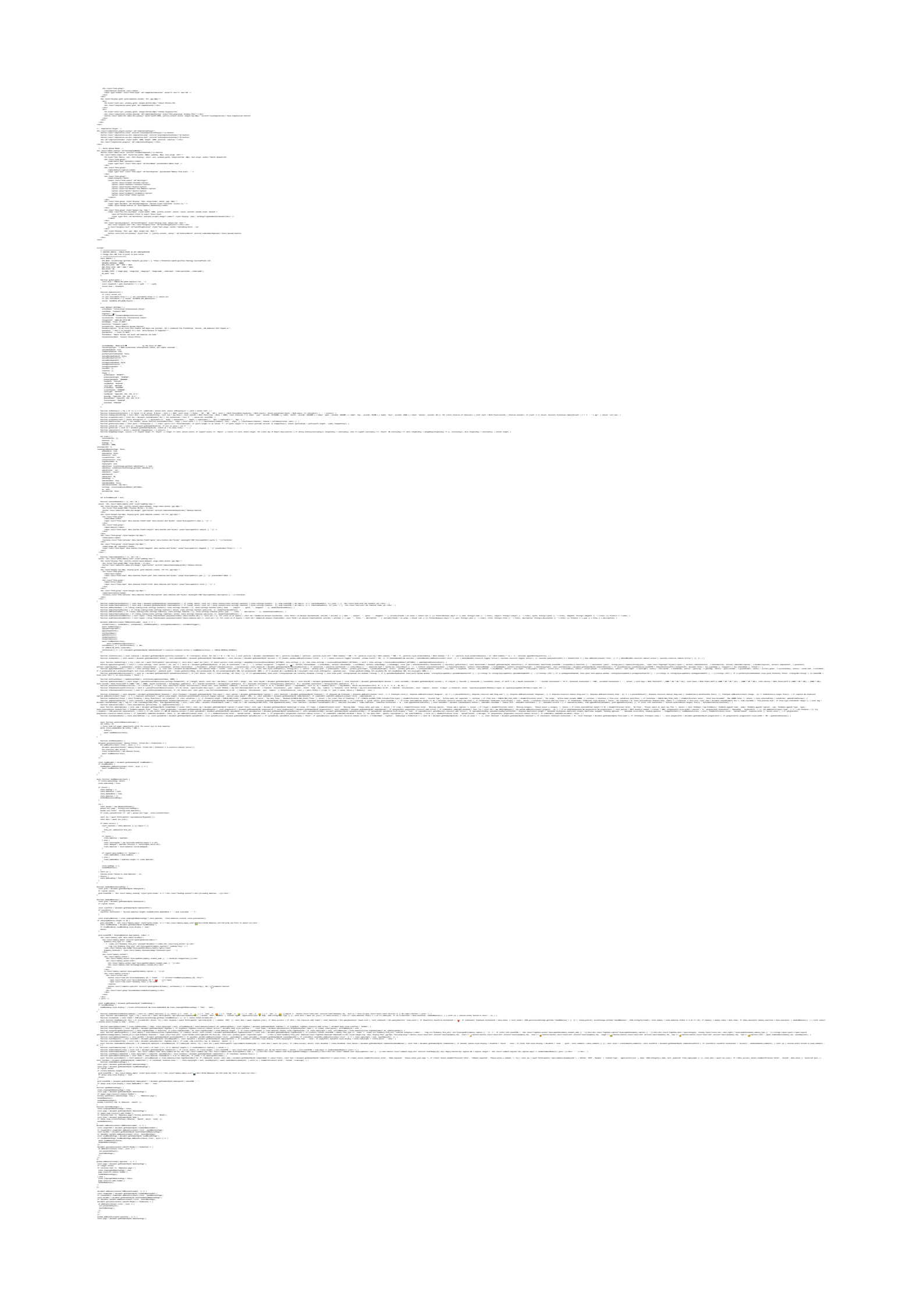
Jordan Arnold

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Tabbawala 360 | Interactive Embodiment Research</title>
  <meta name="description" content="Tabbawala 360 — Dance and Charlie cut bamboo together" />

  <link rel="preconnect" href="https://fonts.gstatic.com" />
  <link rel="stylesheet" href="https://fonts.google.com" onload="void" />
  <noscript><link rel="stylesheet" href="https://fonts.google.com" />
  <meta name="theme-color" content="#0d0d0d" />

  <style>
    /* ═══ ═══
    :root {
      --primary-gold: #D4AF37;
      --primary-gold-light: #F0D060;
      --primary-gold-dark: #B8962E;
      --bg-deep: #0a0a0a;
      --bg-card: #1a1a1a;
      --bg-light: #F7F5E8;
      --text-dark: #1a1a1a;
      --text-gray: #666666;
      --text-light: #F7F5E8;
      --text-muted: #a1a1a1;
      --glass-bg: rgba(26, 26, 26, 0.7);
      --glass-bg-light: rgba(255, 255, 255, 0.7);
      --glass-bg-dark: rgba(0, 0, 0, 0.5);
      --glass-gold: #D4AF37;
      --shadow-sm:
        0 1px 2px rgba(0, 0, 0, 0.5),
        0 0 0 1px rgba(212, 175, 55, 0.1);
      --shadow-md:
        0 4px 6px rgba(0, 0, 0, 0.5),
        0 0 0 1px rgba(212, 175, 55, 0.2);
      --shadow-lg:
        0 10px 15px rgba(0, 0, 0, 0.5),
        0 0 0 1px rgba(212, 175, 55, 0.3);
      --shadow-gold: 0 0 20px rgba(212, 175, 55, 0.3);
      --shadow-gold-strong: 0 0 40px rgba(212, 175, 55, 0.5);
      --gradient-gold: linear-gradient(135deg, #D4AF37 0%, #F0D060 50%, #B8962E 100%);
      --gradient-dark: linear-gradient(135deg, #1a1a1a 0%, #0a0a0a 100%);
      --gradient-hero: linear-gradient(135deg, rgba(26, 26, 26, 0.9) 0%, rgba(10, 10, 10, 0.8) 100%);
      --font-sans: "Inter", "Noto Sans", sans-serif;
      --font-mono: "Fira Code", "Fira Mono", monospace;
      --font-size: "Noto Sans Bengali", sans-serif;
      --font-origin: "Noto Sans Bengali", sans-serif;

      --section-padding: 100px;
      --container-max: 1200px;

      --radius-xsf: 0 0px 0px rgba(0, 0, 0, 0.);
      --radius-gold: 0 0px 0px rgba(212, 175, 55, 0.);
      --radius-gold-strong: 0 0px 0px rgba(212, 175, 55, 0.);
    }

    /* Prefers-reduced-motion: disable all animations and transitions */
    @media (prefers-reduced-motion: reduce) {
      *,
      *::before,
      *::after {
        animation-duration: 0.01ms !important;
        animation-iteration-count: 1 !important;
        transition-duration: 0.01ms !important;
        scroll-behavior: auto;
      }
    }

    /* ═══ ═══
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    html {
      font-family: var(--font-sans);
      scroll-behavior: smooth;
      -webkit-text-size-adjust: 100%;
      -moz-text-size-adjust: 100%;
      text-size-adjust: 100%;
      -webkit-font-smoothing: ;
      -moz-osx-font-smoothing: grayscale;
      text-rendering: optimizeLegibility;
      overflow-x: hidden;
      overflow-y: scroll;
    }

    body {
      -webkit-scrollbar {width: 10px; }
      -webkit-scrollbar-track { background: var(--bg-deep); }
      -webkit-scrollbar-thumb { background: var(--gold-dark); border-radius: 5px; }
      -webkit-scrollbar-thumb:hover { background: var(--primary-gold-dark); }

      display:block flex { display:block flex;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1);opacity:1;transform:translateY(0px);-webkit-backdrop-filter:blur(0px);backdrop-filter:blur(0px); }
      display:block flex:hover { opacity:0.8;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1); }
      display:block flex:active { opacity:0.7;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1);transform:translateY(0.0px); }
      display:block inline-flex { display:block inline-flex;transition:0.3s ease;cubic-bezier(0.4,0.0,0.2,1);-webkit-backdrop-filter:blur(0px);backdrop-filter:blur(0px);-webkit-text-fill-color:rgba(0,0,0,0.);-webkit-text-stroke-color:rgba(0,0,0,0.);-webkit-text-stroke-width:0.0px;-webkit-mask-image:linear-gradient(rgba(0,0,0,0.),rgba(0,0,0,0.);-webkit-mask-size:100% 100%;-webkit-mask-position:0.0% 0.0px;-webkit-mask-repeat:no-repeat;-webkit-mask-clip:content-box;-webkit-mask-composite:source-over;-webkit-mask-mode:alpha;-webkit-mask-origin:content-box
```

```
    .separation.photo.shot:hover { border-color: rgba(255,179,71,0.5); }
    .separation.photo.selected { border-color: var(--primary-gold); }
    .separation.photo.shot.selected { border-color: var(--primary-gold); }
    .separation.photo .photo-check { position: absolute; top: 8px; right: 8px; width: 40px; height: 40px; background: var(--primary-gold); border-radius: 50%; display: none; align-items: center; justify-content: center; color: var(--navy-dark); font-weight: bold; }
    .separation.photo.selected .photo-check { display: flex; }

    .separation.photo.photo-placeholder { display: flex; gap: 10px; overflow-x: auto; padding: 15px 0; min-height: 140px; background: rgba(0,0,0,0.3); border-radius: 8px; margin: 15px 0; }
    .separation.photo.photo-placeholder:hover { color: var(--primary-gold); }
    .separation.photo.photo-placeholder.shot { min-width: 80px; height: 80px; background: rgba(255,179,71,0.1); border: 1px solid rgba(255,179,71,0.3); }
    .separation.photo.photo-placeholder.shot:hover { transform: scale(1.05); }
    .separation.photo.photo-placeholder.shot.selected { background: rgba(255,179,71,0.2); }
    .separation.photo.photo-placeholder.shot.selected::before { content: "✓"; position: absolute; top: 5px; right: 5px; width: 20px; height: 20px; background: var(--primary-gold); color: var(--navy-dark); border-radius: 50%; display: flex; align-items: center; justify-content: center; font-size: 12px; }

    /* Action grid styles */
    .action.selectable { cursor: pointer; border: 2px dashed transparent; padding: 2px 0px; border-radius: 4px; transition: var(--transition-smooth); }
    .action.selectable:hover { background: rgba(255,179,71,0.1); border-color: rgba(255,179,71,0.3); }
    .action.selected { background: rgba(255,179,71,0.2); border: 2px solid var(--primary-gold); }
    .action.edit-input { background: rgba(255,179,71,0.1); border: 1px solid var(--primary-gold); color: var(--cream); text-right; padding: 2px 0px; border-radius: 4px; font-family: inherit; font-size: inherit; width: 100%; }
    </style>

    <style_ref_finish_historical_web_layout_pdf_fr_1
    .header_fixed_fixed_fixed_web_layout_pdf_fr_1 { z-index:dashboard; .wrt: transparent; }
    .header_fixed_fixed_fr_1 {
    .header_fixed_fixed_fr_1: background:rgba(0,0,0,0.0); { z-index:10000; display:none;
    .header_fixed_fixed_fr_1:
    .header_fixed_scanning_active{ display:block; }
    .header_fixed_fixed_fr_1:
    position:fixed; border-radius:0px; box-shadow:0,0,0,0:0px; rgba(0,0,0,0.0); { z-index:10000; position:absolute;top;
    transition:all 0.3s ease;
    }
    .header_fixed_nav{
    .header_fixed_fixed_fr_1:0px; width:calc(100%-12px); rgba(0,0,0,0.0); background:var(--navy_dark);
    border:1px solid var(--glass_border); .header_fixed_fixed_fr_1:0px; .padding:15px;
    .header_fixed_fixed_fr_1:0px; .padding:15px;
    }
    .header_fixed_fixed_fr_1:0px;.header_fixed_fixed_fr_1:0px; .header_fixed_fixed_fr_1:0px; font_family:var(--font_display);
    .header_fixed_fixed_fr_1:var(--text_secondary);.header_fixed_fixed_fr_1:0px;.header_fixed_fixed_fr_1:0px;
    .
```

```
    color: var(--neg-dark);
    border-radius: 10px;
    cursor: pointer;
    text-weight: 700;
    transition: all 0.2s cubic-bezier(0.4,0.0,0.1);

  .tagfilter,.dashed-tree { transform: scale(1.0); }

  /* Subpage */
  .subpage-header{
    display:flex;
    gap:16px;
    justify-content:space-between;
    flex-wrap:wrap;
    gap: 16px;
    margin-bottom: 16px;

  .subpage-title{
    display:flex;
    gap:12px;
    flex-wrap:wrap;
    flex-basis: var(--text-display);
    text-weight: 700;
    color: var(--primary-grad);

  .subpage-dashed{
    display:flex;
    gap:12px;
    flex-wrap:wrap;
    text-weight: 700;
    color: var(--dashed-grad);
    margin-bottom: 16px;

  .subpage-form{
    display:flex;
    gap:12px;
    flex-wrap:wrap;

  .subpage-form-row{
    display:grid;
    grid-template-columns: 1fr;
    gap: 16px;

  .subpage-form-actions{
    display:flex;
    gap:12px;
    flex-wrap:wrap;
    flex-basis: var(--text-display);
    justify-content:flex-end;

  .subpage-table-wrap{
    overflow-x: hidden;
    text-weight: 700;
    background: rgba(0,0,0,0.4);
    border-radius: 10px;
    gap: 16px;

  .subpage-tag{
    display:flex;
    gap:12px;
    flex-wrap:wrap;
    text-weight: 700;
    color: var(--text-dashed);
    text-weight: 700;
    text-weight: 700;
    text-weight: 700;
```

```
  ,'ucrt','ijg',sizeof(FMT/\*)\*\*\*
```



```
if (!page) return;
if (location.hash !== '#admin-page') {
  page.classList.remove('active');
  page.classList.remove('hidden');
  return;
} else {
  page.classList.remove('hidden');
  page.classList.add('active');
}

(function() {
  const adminLoginForm = document.getElementById('admin-login-form');
  const adminLoginError = document.getElementById('admin-login-error');
  const adminLoginBtnTxt = document.getElementById('admin-login-btn-text');
  const adminLoginBtnLoad = document.getElementById('admin-login-btn-loading');

  function showAdminLoginLoading(isLoading) { if (!adminLoginBtnTxt || !adminLoginBtnLoad) return; if (isLoading) { adminLoginBtnTxt.style.display = 'none'; adminLoginBtnLoad.style.display = 'inline'; } else { adminLoginBtnTxt.style.display = 'inline'; adminLoginBtnLoad.style.display = 'none'; } }
  function showAdminLoginError(msg) { if (!adminLoginError) return; adminLoginError.textContent = msg; adminLoginError.style.display = msg ? 'block' : 'none'; }

  if (adminLoginForm) { adminLoginForm.addEventListener('submit', async function(e) { e.preventDefault(); const email = document.getElementById('admin-email').value.trim(); const password = document.getElementById('admin-password').value; if (!email || !password) { showAdminLoginError('Please enter email and password'); return; } showAdminLoginError(''); showAdminLoginLoading(true); try { const res = fetch('/api/auth/login', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ email, password }), credentials: 'same-origin' }); const data = (res && typeof res.then === 'function') ? null : null; if (!data || !data.success) { showAdminLoginError(data && data.error ? data.error : 'Invalid email or password'); return; } if (data.user && data.user.role !== 'admin') { showAdminLoginError('Access denied. Admin accounts only.'); return; } window.location.reload(); } catch (e) { showAdminLoginError('Network error. Please try again.'); } finally { showAdminLoginLoading(false); } }); }

  function adminFetch(path, opts) { opts = opts || {}; const headers = opts.headers || {}; headers['Content-Type'] = headers['Content-Type'] || 'application/json'; return fetch(path, { method: opts.method || 'GET', headers: headers, body: opts.body, credentials: 'same-origin' }).then(function(r) { return r.text().then(function(t) { var d = null; try { d = t ? JSON.parse(t) : null; } catch (e) { d = null; } return { status: r.status, ok: r.ok, data: d }; }); }); }

  function adminEscape(s) { if (s == null) return ''; return String(s).replace(/&/g, '&amp;').replace(/<|>|"|'/g, function(c) { return { '<': '&lt;', '>': '&gt;', '&': '&amp;', '"': '&quot;', '\'': '&#39;' }[c]; }); }

  function adminFmtDate(iso) { if (!iso) return ''; var d = new Date(iso); if (isNaN(d.getTime())) return iso; return d.toLocaleDate
```

```
    const data = await dec.join();

    if (!data.current || !data.lastUpdate || !data.lastUpdate.length) {
      container.style.display = 'none';
      return;
    }

    container.style.display = 'block';
    var latestMTM = data.lastUpdate[data.lastUpdate.length - 1];
    var html = '<div>';
    html += '<div>';
    html += '<div>';
      if (!isNaN(latestMTM.ts)) {
        var d = new Date(latestMTM.ts);
        if (!isNaN(d.getTime())) {
          var ago = Math.floor((Date.now() - d.getTime()) / 60000);
          if (ago < 1) ago = 0;
          if (ago < 60) {
            html += '<span>' + ago + ' min ago</span>';
          } else {
            var hours = Math.floor(ago / 60);
            var mins = ago % 60;
            if (hours < 24) {
              html += '<span>' + hours + 'h ' + mins + 'm ago</span>';
            } else {
              var days = Math.floor(hours / 24);
              var rhours = hours % 24;
              html += '<span>' + days + 'd ' + rhours + 'h ago</span>';
            }
          }
        }
      }
      html += '<span>' + latestMTM.ts + '</span>';
    }
    html += '</div>';
    html += '<div>';
    var keys = Object.keys(latestMTM);
    for (var i = 0; i < keys.length; i++) {
      var k = keys[i];
      if (k == 'ts') continue;
      var v = latestMTM[k];
      if (typeof v == 'object') v = JSON.stringify(v);
      if (v == null) v = '';
      if (v == '') v = '-';
      html += '<div>';
      html += '<span>' + k + '</span>';
      html += '<span>' + v + '</span>';
      html += '</div>';
    }
    html += '</div>';
    html += '</div>';
    container.innerHTML = html;
  } catch (e) {
    container.innerHTML = '<div>Error fetching latest MTM: ' + e.';
    container.style.display = 'block';
  }
}

// ============================================================================
// WATER DRAIN CHARTS
// ============================================================================
function updateWaterDrainCharts() {
  var startVal = document.getElementById('chartStart').value;
  var endVal = document.getElementById('chartEnd').value;
  var qs = '';
  if (startVal) qs += '?start=' + encodeURIComponent(new Date(startVal).toISOString());
  if (endVal) qs += (qs ? '&' : '?') + 'end=' + encodeURIComponent(new Date(endVal).toISOString());
  var ts = Date.now();
  var wrap = document.getElementById('waterDrainImgWrap');
  if (wrap) wrap.innerHTML = '<div>Loading...</div>';
  var ldEl = document.getElementById('waterDrainLoading');
  if (ldEl) ldEl.textContent = 'Loading...';
  var ldEl2 = document.getElementById('tankVolumeLoading');
  if (ldEl2) ldEl2.textContent = 'Loading...';
  var ldEl3 = document.getElementById('tankVolumeImgWrap');
  if (ldEl3) ldEl3.innerHTML = '<div>Loading...</div>';
  var ldEl4 = document.getElementById('tankVolumeLatest');
  if (ldEl4) ldEl4.textContent = '';
  var ldEl5 = document.getElementById('tankVolumeTotal');
  if (ldEl5) ldEl5.textContent = '';
  var ldEl6 = document.getElementById('tankVolumeTotalRate');
  if (ldEl6) ldEl6.textContent = '';
  var ldEl7 = document.getElementById('tankVolumeTotalDrain');
  if (ldEl7) ldEl7.textContent = '';
  var ldEl8 = document.getElementById('tankVolumeTotalDrainRate');
  if (ldEl8) ldEl8.textContent = '';
  var ldEl9 = document.getElementById('tankVolumeTotalDrainDuration');
  if (ldEl9) ldEl9.textContent = '';
  var ldEl10 = document.getElementById('tankVolumeTotalDrainCount');
  if (ldEl10) ldEl10.textContent = '';
  var ldEl11 = document.getElementById('tankVolumeTotalDrainAvg');
  if (ldEl11) ldEl11.textContent = '';
  var ldEl12 = document.getElementById('tankVolumeTotalDrainAvgRate');
  if (ldEl12) ldEl12.textContent = '';
  var ldEl13 = document.getElementById('tankVolumeTotalDrainAvgDuration');
  if (ldEl13) ldEl13.textContent = '';
  var ldEl14 = document.getElementById('tankVolumeTotalDrainAvgCount');
  if (ldEl14) ldEl14.textContent = '';
  var ldEl15 = document.getElementById('tankVolumeTotalDrainAvgDurationCount');
  if (ldEl15) ldEl15.textContent = '';
  var ldEl16 = document.getElementById('tankVolumeTotalDrainAvgDurationRate');
  if (ldEl16) ldEl16.textContent = '';
  var ldEl17 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCount');
  if (ldEl17) ldEl17.textContent = '';
  var ldEl18 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRate');
  if (ldEl18) ldEl18.textContent = '';
  var ldEl19 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCount');
  if (ldEl19) ldEl19.textContent = '';
  var ldEl20 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRate');
  if (ldEl20) ldEl20.textContent = '';
  var ldEl21 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCount');
  if (ldEl21) ldEl21.textContent = '';
  var ldEl22 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRate');
  if (ldEl22) ldEl22.textContent = '';
  var ldEl23 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCount');
  if (ldEl23) ldEl23.textContent = '';
  var ldEl24 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCountRate');
  if (ldEl24) ldEl24.textContent = '';
  var ldEl25 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCountRateCount');
  if (ldEl25) ldEl25.textContent = '';
  var ldEl26 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCountRateCountRate');
  if (ldEl26) ldEl26.textContent = '';
  var ldEl27 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCountRateCountRateCount');
  if (ldEl27) ldEl27.textContent = '';
  var ldEl28 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCountRateCountRateCountRate');
  if (ldEl28) ldEl28.textContent = '';
  var ldEl29 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCountRateCountRateCountRateCount');
  if (ldEl29) ldEl29.textContent = '';
  var ldEl30 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCountRateCountRateCountRateCountRate');
  if (ldEl30) ldEl30.textContent = '';
  var ldEl31 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCountRateCountRateCountRateCount');
  if (ldEl31) ldEl31.textContent = '';
  var ldEl32 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCountRateCountRateCountRateCountRate');
  if (ldEl32) ldEl32.textContent = '';
  var ldEl33 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCountRateCountRateCountRateCount');
  if (ldEl33) ldEl33.textContent = '';
  var ldEl34 = document.getElementById('tankVolumeTotalDrainAvgDurationRateCountRateCountRateCountRateCountRateCountRateCountRateCountRate');
  if
```

```
  image_size[0] = 1;
  var bg = document.createElement('bg');
  bg.id = 'bg-layer';
  bg.style.opacity = '0';
  bg.style.transition = 'opacity 2s';
  image.appendChild(bg);
  var caption = document.createElement('p');
  caption.style.transition = 'transform 0.4s ease, opacity .8s ease';
  caption.id = 'caption';
  caption.style.transform = 'scale(.95) translateX(0px,-14px)';

  canvas.textContent = 'Lord Rain Mode | Ctrl+Shift+R: toggle';
  canvas.setAttribute('aspect', 'Math.min(window.innerWidth / 9, window.innerHeight / 16)');
  canvas.footer.textContent = canvasHeight(){
    canvas.footer.textContent = canvasHeight(){
      canvas.footer.textContent = canvasHeight(){ 1 % canvas.footer.textContent.length;
      canvas.footer.textContent = '';
    }, 1000);
  }, 1000);
}

function loadMoreCards(n, cards){
  var layer = document.getElementById('ambiguity');
  var cx = window.innerWidth / 2;
  var cy = window.innerHeight / 2;
  var rx = window.innerWidth * 0.42;
  var ry = window.innerHeight * 0.42;
  var i, c, angle, x, y, el;
  for (i = 0; i < n; i++) {
    c = cards[Math.floor(Math.random() * cards.length)];
    el = document.createElement('div');
    el.textNode = 'ambiguity';
    layer.appendChild(el);
    angle = (i / n) * 6.283185307179586 + (Math.random() - 0.5) * 0.4;
    x = cx + rx * Math.cos(angle) + (Math.random() - 0.5) * 60;
    y = cy + ry * Math.sin(angle) + (Math.random() - 0.5) * 60;
    el.style = {
      'left': x + 'px',
      'top': y + 'px',
      'transform': 'translate(-50%,-50%) rotate(' + (Math.random() * 360 - 180) + 'deg)',
      'animation-delay': (Math.random() * 6) + 's',
      'animation-duration': (4 + Math.random() * 4) + 's',
      'opacity': (0.3 + Math.random() * 0.7).toFixed(2),
      'color': c,
      'text-shadow': '0 0 8px ' + c + ', 0 0 16px ' + c,
      'font-size': (Math.random() * 14 + 10) + 'px',
      'pointer-events': 'none',
    };
  }
}

function initCanvas(){
  var canvas = document.getElementById('canvas');
  if (!canvas) return;
  var ctx = canvas.getContext('2d');
  var W, H, dpr = 1, cols, drops, speed, chars = '0123456789ABCDEF';
  var fontSize = 14, trailAlpha = 0.05;
  var lastTime = 0, frameMs = 1000 / 30;
  var resize = function(){
    var parent = canvas.parentNode;
    var rect = parent ? parent.getBoundingClientRect() : {width: 0, height: 0};
    var cssW = rect.width || window.innerWidth;
    var cssH = rect.height || window.innerHeight;
    dpr = (window.devicePixelRatio || 1);
    if (dpr > 2) dpr = 2;
    canvas.style.width = cssW + 'px';
    canvas.style.height = cssH + 'px';
    canvas.width = Math.floor(cssW * dpr);
    canvas.height = Math.floor(cssH * dpr);
    W = canvas.width;
    H = canvas.height;
    var n = Math.floor(W / (fontSize * dpr));
    if (n < 1) n = 1;
    cols = n;
    drops = [];
    speed = [];
    var i = 0;
    while (i < cols) {
    drops[i] = Math.random() * -(H / (fontSize * dpr));
    speed[i] = 0.5 + Math.random() * 1.5;
    i = i + 1;
    }
  };
  var draw = function(){
    var i = 0;
    var y, ch, head, tail, alpha, hue, sat, light, fx, fy, fsize, fhue, fsat, flight, fdrift, fspeed, fphase, ftrail, fcount, f, ftail, ftail2, ftail3, ftail4, ftail5, ftail6, ftail7, ftail8, ftail9, ftail10, ftail11, ftail12, ftail13, ftail14, ftail15, ftail16, ftail17, ftail18, ftail19, ftail20, ftail21, ftail22, ftail23, ftail24, ftail25, ftail26, ftail27, ftail28, ftail29, ftail30, ftail31, ftail32, ftail33, ftail34, ftail35, ftail36, ftail37, ftail38, ftail39, ftail40, ftail41,
```

```
    logo: appendfirstlogline(),
    ver = 2.00 || window.innerheight - 50;
    ver = ver / 2 - 20;
    ver_hidden = 0
    ver_hidden = ver
    # if(ver || ver_hp, hp = 0.85, false);
    phase_state_hp = ver_hp / ver
    phase_state_hp = 1 / ver
    phase_state_hp = phase_state_hp || 1 && phase_state_hp < 1 && phase_state_hp; return;
    requestAnimationFrame(step);
  }
  step();
}

function fitlineseqfrag(lines){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !lines || !lines.length) return;
  phase_lines_hp = lines.length;
  phase_state_hp = 0; phase_state = 0;
  logo.appendChild(frag);
  var ver = 2.00 || window.innerheight - 50;
  ver = ver / 2 - 20;
  ver_hidden = 0
  ver_hidden = ver
  phase_state_hp = 1 / ver
  phase_state_hp = phase_state_hp || 1 && phase_state_hp < 1 && phase_state_hp; return;
  requestAnimationFrame(step);
  step();
}

function fitlineseqfrag(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages || [];
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfter(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelayAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelayAfterDelayAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelayAfterDelayAfterDelayAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelayAfterDelayAfterDelayAfterDelayAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelayAfterDelay(){
  var logo = document.getElementById("logo-appendfirstlineseqfr");
  if(!logo && !window.__stripeMessages) return;
  var lines = window.__stripeMessages;
  var frag = document.createFragment();
  for(var i = 0; i < lines.length; i++){
    var ln = document.createElement("div");
    ln.textContent = lines[i];
    ln.style.opacity = "0";
    ln.style.transition = "opacity 0.3s";
    frag.appendChild(ln);
    (function(el, delay){ setTimeout(function(){ el.style.opacity = "1"; }, delay); })(ln, i * 200);
  }
  logo.appendChild(frag);
}

function pollNewMessagesAfterDelay
```

```
  window.TryInstallPwa = function(){
    var type = document.getElementById('appmkthinkness');
    if (!type) return;
    var data = JSON.parse(+n, string + (!!ver.payload || []).
    var ok = !!data.ok;
    var appmkthinkness;
    if (ok) {
    var w = 640;
    var h = 480;
    var x = (screen.width - w) / 2;
    var y = (screen.height - h) / 2;
    var left = 0;
    var top = 0;
    var lines = [];
    for (i = 0; i < 20; i++) {
      var y = Math.sin(i * 0.4) * 40 + Math.cos(i * 0.7) * 6;
      lines.push('L' + (i * 32) + ' ' + y);
      ctx.fillRect(x, y, 2, 2);
    }
    window.localStorage = {data: '', expr: '', map:
    if (!r || window.innerWidth < 140)
      return;
    } else {
      appmkthinkness(true);
    }
    }
    window.addEventListener('load', [] => {
      var fmt = info.caption || info.text [] "A dainty free-camera.";
      var ok = !!fmt;
      ctx.fillRect(x, y, 2, 2);
      var lines = [];
    });
    setTimeout(function(){
    window.TryInstallPwa = __ORWsqqqm4 | true;
    },
  });
  function makeFakeMedia(info){
    var wrapper = document.createElement('div');
    wrapper.innerHTML = [wrapper.
    var fmt = !!fmt;
    var allowed = !!ok && !!data[0].kind;
    var caption = info.caption || info.text [] "";
    return wrapper;
  }
  function getMbuttonSrc(){
    if (typeof original !== 'function') return __MButtonSrcOrig0.call(this);
    var s = original.call(this);
    if (!s) return s;
    if (s.indexOf('data:image/svg+xml') !== 0) return s;
    var decoded = decodeURIComponent(s.slice('data:image/svg+xml,'.length));
    if (decoded.indexOf('<rect x="0" y="0" width="100" height="100" fill="#ff0000"') !== -1)
      return 'data:image/svg+xml,%3Csvg%20xmlns%3D%27http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%27%20viewBox%3D%270%20-0.5%20100%20100.5%27%20preserveAspectRatio%3D%27xMidYMid%20meet%27%3E%3Cpath%20d%3D%27M%20-5%20-5%20Q%20-5%20-5%20%20-5%20-5%20Z%20M%20105%20-5%20Q%20105%20-5%20%20105%20-5%20Z%20M%20105%20105%20Q%20105%20105%20%20105%20105%20Z%20M%20-5%20105%20Q%20-5%20105%20%20-5%20105%20Z%27%20fill%3D%27%23ff0000%27%2F%3E%3C%2Fsvg%3E';
    return s;
  }
  window.__MButtonSrcOrig0 = __MButtonSrcOrig0 | true;

  window.__MButtonSrcOrig1 = __MButtonSrcOrig1 | true;

  var original = window.HTMLMediaElementSrcGet;
  if (typeof original !== 'function') return __MMediaSrcOrig0;
  var __MMediaSrcOrig0 = function(){
    var s = original.apply(this, arguments);
    if (!s) return s;
    if (s.indexOf('data:image/svg+xml') !== 0) return s;
    var decoded = decodeURIComponent(s.slice('data:image/svg+xml,'.length));
    if (decoded.indexOf('<rect x="0" y="0" width="100" height="100" fill="#ff0000"') !== -1)
      return 'data:image/svg+xml,%3Csvg%20xmlns%3D%27http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%27%20viewBox%3D%270%20-0.5%20100%20100.5%27%20preserveAspectRatio%3D%27xMidYMid%20meet%27%3E%3Cpath%20d%3D%27M%20-5%20-5%20Q%20-5%20-5%20%20-5%20-5%20Z%20M%20105%20-5%20Q%20105%20-5%20%20105%20-5%20Z%20M%20105%20105%20Q%20105%20105%20%20105%20105%20Z%20M%20-5%20105%20Q%20-5%20105%20%20-5%20105%20Z%27%20fill%3D%27%23ff0000%27%2F%3E%3C%2Fsvg%3E';
    return s;
  };
  window.__MMediaSrcOrig0 = __MMediaSrcOrig0 | true;

  var originalSet = window.HTMLMediaElementSrcSet;
  if (typeof originalSet !== 'function') return __MMediaSrcOrig1;
  var __MMediaSrcOrig1 = function(){
    var v = originalSet.apply(this, arguments);
    var allowed = !!ok && !!data[0].kind;
    var caption = info.caption || info.text [] "";
    var wrapper = document.getElementById('ioffmsrcmbuttonfmbuttonfmbuttonf
```

```
        xlr="https://www.spectrofiberop.org/export/sdbxt/5b07f8ccc-c8f8d8-2d8bc3d8bc2d8ep/sagrar"
        allowfullscreen
        frameborder="0"
      ></iframe>
    )
  }
}

function currentlyOnTrigger$$$$(){
  if (window.popupOpened) return;
  window.popupOpened = true;

  const qualMsgs = [1, 2, 3];
  const qualIdx = 0;
  const currEmbed = document.getElementById('currEmbed');
  const qualBtn = [];
  const text = currEmbed.getAttributeNS(null);
  const lowercaseLine = text.toLowerCase().trim();
  const isVisible = document.getElementById('chlcVisibleSection') && getComputedStyle(document.getElementById('chlcVisibleSection')).display !== 'none';
  if (!isVisible || !currEmbed) return;

  window.popupOpened = true;
  setTimeout(function() {
    document.getElementById('qualifyPopup').classList.add('active');
    document.getElementById('qualifyOverlay').classList.add('active');
    updateQualifyDisplay();
  }, 300);

  window.addEventListener('scroll', updateMap, { passive: true });
  setTimeout(updateMap, 400);
}

function hasSeenVisionBefore(){
  const stored = document.getElementById('ls-seenBefore');
  if (typeof window.lsSeenBefore !== 'boolean') return false;
  return window.lsSeenBefore;
  if (getEmbedTo.getAttribute('data-seen') == 'true')
  return false;
}

const updateUsage$ = window.trackTypingUsage$
  || function(){ if (typeof window.__debugLog$ !== 'function') console.log('[usage]', arguments); };
  window.trackTypingUsage$ = function(){
    const lsVal = document.getElementById('ls-seenBefore');
    if (lsVal && lsVal.textContent.trim() == 'true')
    return;
    var bodyEl = document.getElementById('ls-seenBefore');
    if (bodyEl && bodyEl.textContent.trim() == 'true')
    return;
    var lsNewVal = 'true';
    if (typeof window.__debugLog$ !== 'function') console.log('[usage] ls-seenBefore set to true');
    updateUsage$();
  };

function initializeVisionSection(){
  var seenBefore = hasSeenVisionBefore();
  if (typeof window.lsSeenBefore !== 'boolean') return;
  var isFirstVisit = !seenBefore;
  var currentEmbed = document.getElementById('currEmbed');
  var qualifyBtn = document.getElementById('qualifyBtn');
  if (currentEmbed && currentEmbed.textContent.trim() == 'true')
  return;
  if (!isFirstVisit && !window.lsSeenBefore)
  return;
  if (typeof window.showNotification !== 'function') {
    console.log('[vision] showNotification not defined');
    return;
  }
  var delayMs = isFirstVisit ? 1500 : 800;
  setTimeout(function() {
    if (!document.getElementById('currEmbed').textContent.trim() == 'true')
    return;
    var msgs = [
      'Welcome back. Ready to continue?',
      'Welcome back. Your vision is waiting.',
      'Welcome back. Pick up where you left off?'
    ];
    var randomMsg = msgs[Math.floor(Math.random() * msgs.length)];
    if (typeof console.log !== 'undefined') console.log('[vision] showing notification:', randomMsg);
    window.showNotification(randomMsg, 'info');
    if (isFirstVisit && !window.lsSeenBefore
    && typeof window.updateUsage$ !== 'function') {
      var lsNewVal = 'true';
      if (typeof console.log !== 'undefined') console.log('[vision] ls-seenBefore set to true');
      updateUsage$();
    }
  }, delayMs);
}

function initializeVisionSection$$$$(){
  if (typeof window.__initVisionSection$$$$ !== 'function') {
    if (typeof console.log !== 'undefined') console.log('[vision] initializeVisionSection$$$$ not defined');
    return;
  }
  var result = window.__initVisionSection$$$$();
  if (typeof console.log !== 'undefined') console.log('[vision] initializeVisionSection$$$$ result:', result);
  return result;
}

window.initializeVisionSection$$$$ = initializeVisionSection$$$$;

function initializeTypingAnimation(){
  var typingEl = document.getElementById('typingAnimation');
  if (typeof console.log !== 'function') console.log('[typing] initializeTypingAnimation');
  if (!typingEl
  || typeof window.__typingAnimationInit$$$$ !== 'function')
  return;
  var isVisible = typingEl.offsetWidth > 0 && typingEl.offsetHeight > 0;
  if (!isVisible
  && typeof console.log !== 'function')
  console.log('[typing] typing element not visible, skipping');
  var result = window.__typingAnimationInit$$$$();
  if (typeof console.log !== 'function')
  console.log('[typing] animation initialized:', result);
  return result;
}

window.initializeTypingAnimation = initializeTypingAnimation;

function initializeVisionSection$$$$(){
  if (typeof console.log !== 'function') console.log('[vision] initializeVisionSection$$$$');
  var isVisible = document.getElementById('chlcVisibleSection');
  var currentEmbed = document.getElementById('currEmbed');
  var qualifyBtn = document.getElementById('qualifyBtn');
  if (typeof console.log !== 'function')
  console.log('[vision] chlcVisibleSection:', isVisible, 'currEmbed:', currentEmbed, 'qualifyBtn:', qualifyBtn);
  if (isVisible
  && typeof window.showNotification !== 'function') {
    if (typeof console.log !== 'function')
    console.log('[vision] showNotification not defined, skipping');
    return;
  }
  var delayMs = 1500;
  if (typeof console.log !== 'function')
  console.log('[vision] scheduling notification in', delayMs, 'ms');
  setTimeout(function() {
    if (typeof console.log !== 'function')
    console.log('[vision] timeout fired');
    var currentEmbedCheck = document.getElementById('currEmbed');
    if (currentEmbedCheck
    && currentEmbedCheck.textContent.trim() == 'true')
    {
      if (typeof console.log !== 'function')
      console.log('[vision] currEmbed is true, skipping');
      return;
    }
    var isVisibleCheck = document.getElementById('chlcVisibleSection');
    if (!isVisibleCheck
    && typeof console.log !== 'function')
    console.log('[vision] chlcVisibleSection not visible, skipping');
    var msgs = [
      'Welcome back. Ready to continue?',
      'Welcome back. Your vision is waiting.',
      'Welcome back. Pick up where you left off?'
    ];
    var randomMsg = msgs[Math.floor(Math.random() * msgs.length)];
    if (typeof console.log !== 'function')
    console.log('[vision] showing notification:', randomMsg);
    window.showNotification(randomMsg, 'info');
    if (typeof console.log !== 'function')
    console.log('[vision] notification shown');
  }, delayMs);
}

window.initializeVisionSection$$$$ = initializeVisionSection$$$$
```

```
        </div>
        <div>
          <br id="fortune.globe.start">
          <br id="dotsandsquares">
          <br id="dotsandsquares">/*!</div>
          <br id="legend.pref.back">Schwarz-Langer Map Map */
          <br id="legend.pref.back">Schwarz-Langer-Map/
          <br id="legend.pref.back">Schwarz-Langer-Map/
          <br id="legend.pref.back">Schwarz-Langer-Map/
          <br id="legend.pref.back">Schwarz-Langer-Map/
        </div>
        <br />
        <br />
        <br />
```

```
  color:var(--text-muted);
  font:var(--mono);
  text-align:center;
}
.mv-empty{grid-column:1/-1;
  display:grid;
  place-items:center;
  gap:12px;
  padding:40px;
  text-center;
}
.mv-fleet-mini-grid{
  display:grid;
  grid-template-columns:repeat(4,1fr);
  gap:8px;
  max-height:200px;
  overflow:hidden;
}
.mv-fleet-mini-card{
  background:var(--bg-card);
  border:1px solid rgba(255,255,255,.06);
  padding:6px 10px 10px;
  clip-path:polygon(0 0,100% 0,100% 100%,0 100%);
}
.mv-fleet-mini-card.active{
  border:1px solid rgba(255,255,255,.12);
  background:var(--bg-card-hover);
}
.mv-fleet-mini-name{
  position:fixed;
  top:0;
  left:0;
  z-index:9999;
  pointer-events:none;
  opacity:0;
  transform:translate(-100%,-100%);
  transition:opacity .2s ease;
  text-shadow:0 0 8px rgba(0,0,0,.8);
}
.mv-fleet-mini-name.visible{
  opacity:1;
  transform:translate(-100%,-100%);
}
.mv-fleet-mini-name.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade{
  opacity:0;
  transition:opacity .1s ease;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.visible{
  opacity:1;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.hidden{
  opacity:0;
}
.mv-fleet-mini-name.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.fade.
```

```
  if (!maybe || typeof x !== 'object\nan') return;
  if (!Array\nIsArray\nBuffer\n
  if (!Array\nIsArray(x\n\n\n
```

```
import debug = get_prio(debugmode, 'debugs:jpa')
import logfile = get_prio(debugmode, 'logfile:jpa')
import settings(ref = get_prio(debugmode, 'settings:jpa')
import cmdline(ref = get_prio(debugmode, 'cmdline:jpa')
import commands(ref = get_prio(debugmode, 'commands:jpa')
import cmdhelp(ref = get_prio(debugmode, 'cmdhelp:jpa')
import authtoken(ref = get_prio(debugmode, 'auth:jpa')
import authcmd(ref = get_prio(debugmode, 'authcmd:jpa')
import stopwatch(ref = get_prio(debugmode, 'stopwatch:jpa')
import stopwatchcmd(ref = get_prio(debugmode, 'stopwatch_cmd:jpa')
import tasks(ref = get_prio(debugmode, 'tasks:jpa')
import tasksdatabase(ref = get_prio(debugmode, 'tasks_db:jpa')
import tasksdbcmd(ref = get_prio(debugmode, 'tasks_db_cmd:jpa')
import documentation(ref = get_prio(debugmode, 'documentation:jpa')

function initdatabase() {
  if (!fs.existsSync(jpa.DB_DIR)) {
    fs.writeFileSync(jpa.DB_DIR, JSON.stringify({ exercises: [], nutrition: {}, meals: {}});
    console.log("Created exercise database.")
  }
  if (!fs.existsSync(jpa.CONFIGFILE)) {
    fs.writeFileSync(jpa.CONFIGFILE, JSON.stringify({ exercises: [] } ), null, 4));
    console.log("Created exercise configuration.")
  }
  if (!fs.existsSync(jpa.SETTINGSFILE)) {
    fs.writeFileSync(jpa.SETTINGSFILE, JSON.stringify({ settings: {} } ), null, 4));
    console.log("Created settings database.")
  }
  if (!fs.existsSync(jpa.COMMANDFILE)) {
    fs.writeFileSync(jpa.COMMANDFILE, JSON.stringify({ commands: {} } ), null, 4));
    console.log("Created commands database.")
  }
  if (!fs.existsSync(jpa.MEASURESFILE)) {
    fs.writeFileSync(jpa.MEASURESFILE, JSON.stringify({ measures: [] } ), null, 4));
    console.log("Created measures database.")
  }
  if (!fs.existsSync(jpa.USERFILE)) {
    fs.writeFileSync(jpa.USERFILE, JSON.stringify({ users: [] } ), null, 4));
    console.log("Created users database.")
  }
  // admin users database
  if (!fs.existsSync(jpa.ADMINFILE)) {
    console.log("Creating admin database...")
    fs.writeFileSync(jpa.ADMINFILE, JSON.stringify(
      {
        "users": [
          {
            "id": "admin",
            "name": "Super Admin",
            "email": "",
            "password": "$2b$10$jpa$admin",
            "totpSecret": null,
            "totpEnabled": false,
            "apiKeyHash": null,
            "apiKeyPrefix": null,
            "permissions": {
              "dashboard": true,
              "users": true,
              "groups": true,
              "permissions": true,
              "settings": true,
              "logs": true,
              "backup": true,
              "system": true,
              "apikeys": true,
              "tasks": true,
              "notes": true,
              "calendar": true,
              "reminders": true,
              "contacts": true,
              "expenses": true,
              "nutrition": true,
              "meals": true,
              "measures": true,
              "exercises": true
            }
          }
        ]
      } , null, 4));
    console.log("Created admin database with super admin.")
  }
  if (!fs.existsSync(jpa.NOTESFILE)) {
    fs.writeFileSync(jpa.NOTESFILE, JSON.stringify({ categories: [], notes: [] } ), null, 4));
    console.log("Created notes database.")
  }

}
if (!fs.existsSync(jpa.CALENDARFILE)) {
  fs.writeFileSync(jpa.CALENDARFILE, JSON.stringify({ events: [] } ), null, 4));
  console.log("Created calendar database.")
}
if (!fs.existsSync(jpa.REMINDERSFILE)) {
  fs.writeFileSync(jpa.REMINDERSFILE, JSON.stringify({ reminders: [] } ), null, 4));
  console.log("Created reminders database.")
}
if (!fs.existsSync(jpa.CONTACTSFILE)) {
  fs.writeFileSync(jpa.CONTACTSFILE, JSON.stringify({ contacts: [], groups: [], emails: [], phones: [], addresses: [], notes: [], other: [] } ), null, 4));
  console.log("Created contacts database.")
}
if (!fs.existsSync(jpa.EXPENSESFILE)) {
  fs.writeFileSync(jpa.EXPENSESFILE, JSON.stringify({ expenses: [], categories: [], subcategories: [], budget: {} } ), null, 4));
  console.log("Created expenses database.")
}

function isMemberOfGroup(fullPath, fullGroup) {
  try {
    return JSON.parse(fs.readFileSync(fullPath, 'utf8').
    .groups.
    .some(g => g
    .trim() ===
    fullGroup);
  }
  catch
  {
    return false;
  }
}
function isMatch(str,pattern, data=''') {
  try {return jpa.minimatch(str,pattern, {nocase:true}); }
  catch { return false; }
}

function isAdmin() {
  return isMemberOfGroup(jpa.ADMINFILE, 'admins');
}

function isUsersMember() {
  return isMemberOfGroup(jpa.USERFILE, 'users');
}

function isExercisesMember() {
  return isMemberOfGroup(jpa.CONFIGFILE, 'exercises');
}

function isSettingsMember() {
  return isMemberOfGroup(jpa.SETTINGSFILE, 'settings');
}

function isCommandsMember() {
  return isMemberOfGroup(jpa.COMMANDFILE, 'commands');
}

function isMeasuresMember() {
  return isMemberOfGroup(jpa.MEASURESFILE, 'measures');
}

function isTasksMember() {
  return isMemberOfGroup(jpa.TASKSFILE, 'tasks');
}

function isNotesMember() {
  return isMemberOfGroup(jpa.NOTESFILE, 'notes');
}

function isCalendarMember() {
  return isMemberOfGroup(jpa.CALENDARFILE, 'events');
}

function isRemindersMember() {
  return isMemberOfGroup(jpa.REMINDERSFILE, 'reminders');
}

function isContactsMember() {
  return isMemberOfGroup(jpa.CONTACTSFILE, 'contacts');
}

function isExpensesMember() {
  return isMemberOfGroup(jpa.EXPENSESFILE, 'expenses');
}

function isNutritionMember() {
  return isMemberOfGroup(jpa.NUTRITIONFILE, 'nutrition');
}

function isMealsMember() {
  return isMemberOfGroup(jpa.MEALSFILE, 'meals');
}

function isMeasuresMember() {
  return isMemberOfGroup(jpa.MEASURESFILE, 'measures');
}

function isAuthenticated() {
  return isAdmin() || isUsersMember();
}

function isAuthenticatedAdmin() {
  return isAdmin();
}

function isAuthenticatedUsersMember() {
  return isUsersMember();
}

function isAuthenticatedExercisesMember() {
  return isExercisesMember();
}

function isAuthenticatedSettingsMember() {
  return isSettingsMember();
}

function isAuthenticatedCommandsMember() {
  return isCommandsMember();
}

function isAuthenticatedMeasuresMember() {
  return isMeasuresMember();
}

function isAuthenticatedTasksMember() {
  return isTasksMember();
}

function isAuthenticatedNotesMember() {
  return isNotesMember();
}

function isAuthenticatedCalendarMember() {
  return isCalendarMember();
}

function isAuthenticatedRemindersMember() {
  return isRemindersMember();
}

function isAuthenticatedContactsMember() {
  return isContactsMember();
}

function isAuthenticatedExpensesMember() {
  return isExpensesMember();
}

function isAuthenticatedNutritionMember() {
  return isNutritionMember();
}

function isAuthenticatedMealsMember() {
  return isMealsMember();
}

function isAuthenticatedMeasuresMember() {
  return isMeasuresMember();
}

function authenticatedAdmin(req, res) {
  if (!isAuthenticatedAdmin()) {
    res.status(403).send('Forbidden - Admin access required');
    return false;
  }
  return true;
}

function authenticatedUsersMember(req, res) {
  if (!isAuthenticatedUsersMember()) {
    res.status(403).send('Forbidden - Users access required');
    return false;
  }
  return true;
}

function authenticatedExercisesMember(req, res) {
  if (!isAuthenticatedExercisesMember()) {
    res.status(403).send('Forbidden - Exercises access required');
    return false;
  }
  return true;
}

function authenticatedSettingsMember(req, res) {
  if (!isAuthenticatedSettingsMember()) {
    res.status(403).send('Forbidden - Settings access required');
    return false;
  }
  return true;
}

function authenticatedCommandsMember(req, res) {
  if (!isAuthenticatedCommandsMember()) {
    res.status(403).send('Forbidden - Commands access required');
    return false;
  }
  return true;
}

function authenticatedMeasuresMember(req, res) {
  if (!isAuthenticatedMeasuresMember()) {
    res.status(403).send('Forbidden - Measures access required');
    return false;
  }
  return true;
}

function authenticatedTasksMember(req, res) {
  if (!isAuthenticatedTasksMember()) {
    res.status(403).send('Forbidden - Tasks access required');
    return false;
  }
  return true;
}

function authenticatedNotesMember(req, res) {
  if (!isAuthenticatedNotesMember()) {
    res.status(403).send('Forbidden - Notes access required');
    return false;
  }
  return true;
}

function authenticatedCalendarMember(req, res) {
  if (!isAuthenticatedCalendarMember()) {
    res.status(403).send('Forbidden - Calendar access required');
    return false;
  }
  return true;
}

function authenticatedRemindersMember(req, res) {
  if (!isAuthenticatedRemindersMember()) {
    res.status(403).send('Forbidden - Reminders access required');
    return false;
  }
  return true;
}

function authenticatedContactsMember(req, res) {
  if (!isAuthenticatedContactsMember()) {
    res.status(403).send('Forbidden - Contacts access required');
    return false;
  }
  return true;
}

function authenticatedExpensesMember(req, res) {
  if (!isAuthenticatedExpensesMember()) {
    res.status(403).send('Forbidden - Expenses access required');
    return false;
  }
  return true;
}

function authenticatedNutritionMember(req, res) {
  if (!isAuthenticatedNutritionMember()) {
    res.status(403).send('Forbidden - Nutrition access required');
    return false;
  }
  return true;
}

function authenticatedMealsMember(req, res) {
  if (!isAuthenticatedMealsMember()) {
    res.status(403).send('Forbidden - Meals access required');
    return false;
  }
  return true;
}

function authenticatedMeasuresMember(req, res) {
  if (!isAuthenticatedMeasuresMember()) {
    res.status(403).send('Forbidden - Measures access required');
    return false;
  }
  return true;
}

function authenticatedAdmin(req, res) {
  if (!isAuthenticatedAdmin()) {
    res.status(403).send('Forbidden - Admin access required');
    return false;
  }
  return true;
}

function authenticated(req, res) {
  if (!isAuthenticated()) {
    res.status(403).send('Forbidden - Authentication
```

```
)
}
// Upload Memory
app.post('/api/upload', upload.array('files', MAX_FILES), (req, res) => {
  try {
    const files = req.files || [];
    const folder = req.body.folder;

    const settings = getMyPrivateSettings();

    if (!settings.uploadEnabled) {
      return res.status(403).json({ success: false, error: 'Uploads are currently disabled' });
    }

    if (settings.uploadRestrictedMode) {
      const allowedUsers = (settings.uploadAllowedUsers || '').split(',').map(u => u.trim());
      if (!req.session.user.isAdmin(MM) && !req.session?.user.isAdmin && !allowedUsers.includes(req.session.upload_username ||
    }

    if (!files || files.length) {return res.status(400).json({ success: false, error: 'Upload was cancelled or no files' });
    if (!folder || folder.includes('..') || folder.includes('/') || folder.includes('\\') || folder.length > 100) {
    if (!folder || !fs.existsSync(MM.join(MM.resolve(MM.__dirname, '..'), folder).startsWith(MM.resolve(MM.__dirname, '..'
      return res.status(400).json({ success: false, error: 'Invalid folder' });
    }

    if (settings.uploadAuthorizationRequired) {
      const authorizedUsers = (settings.uploadAuthorizedUsers || '').split(',').map(u => u.trim).filter(Boolean);
      if (authorizedUsers.length > 0 && !authorizedUsers.includes(req.session.user.username.toLowerCase())) {
        return res.status(403).json({ success: false, error: 'Upload rejected by profanity filter' });
      }
    }

    const MB = 1024*1024;
    const maxFileSize = 5 * MB;
    const maxTotalSize = 20 * MB;
    const ALLOWED_EXTENSIONS = (settings.uploadAllowedExt || 'jpg,jpeg,png,gif,webp,mp4,webm,mov,mp3,wav,ogg,m4a,pdf,zip,
    const ALLOWED_MIME_PREFIXES = (settings.uploadAllowedMime || 'image/,video/,audio/,application/pdf,application/zip,').

    const MAX_MULTIFILE = 5;
    const TempDir = path.join(process.cwd(), 'tmp_uploads');
    const Jobs = {}; // jobId:{phase,total,done,current,error}

    const safeJoin = (base, target) => {
      const resolved = MM.resolve(MM.join(base, target));
      if (resolved !== MM.resolve(base) && !resolved.startsWith(MM.resolve(base) + MM.sep)) {
        throw new Error('Path traversal detected');
      }
      return resolved;
    };

    const sanitize = (
      id: MM.Number,
      name: MM.String(name).trim(),
      caption: caption.trim().substring(0, 500),
      folder: folder,
      file_path: file.filename,
      thumb_path: thumbnailPath,
      file_size: parseInt(file.size),
      file_type: getFileType(file.mimetype),
      mime_type: file.mimetype,
      upload_date: MM.Date.now(),
      uploaded_by: req.session.user.id,
      ratings: 0,
      favorites: 0,
      views: 0,
      downloads: 0,
      comments: 0,
      created_at: Date(),
      updated_at: Date(),
    );

    MM.updateOne('photos', sanitize());

    MM.saveDatabase();

    setTimeOut(MM,
    );

    if (req.files.length > 10) {MM.Warn(req.files.length + MM.);
      return res.status(400).
      error: 'Maximum upload batch size 10 files',
      max_allowed: 10,
      received: req.files.length,
    });
    }

    console.log(`[ ${req.ip}${req.files?req.files.length:0}files from ${req.session?.user?req.session.user.$:$ip$}size:${req.
    Boolean(req.files && req.files.length > 1 && req.files.length <= 10);

    res.json({
      success: true,
      message: `Successfully uploaded ${req.files?req.files.length:0}files${req.files?req.files.length > 1 ? 'files':'
      uploaded: req.files.length,
      job: req.files[0],
      jobs: req.files,
    });
    } catch (error) {
    console.error('Upload error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Get Uploaded Memories
app.get('/api/uploads', (req, res) => {
  try {
    const MB = 1024*1024;

    const page = Math.max(1, parseInt(req.query.page || '1', 10));
    const limit = Math.min(100, Math.max(1, parseInt(req.query.limit || '20', 10)));
    const offset = (page - 1) * limit;
    const search = req.query.search ? new RegExp(req.query.search) : null;

    let memories = [];
    const allPhotos = MM.find('photos');
    const currentUsers = MM.find('users');

    if (!allPhotos || !currentUsers) {
      return res.status(500).json({ success: false, error: 'Database error' });
    }

    for (const photo of allPhotos) {
      const uploaderUsers = currentUsers.filter(u => u.id === photo.uploaded_by);
      if (uploaderUsers.length > 0) {
        const uploader = uploaderUsers[0];

        if (!photo.is_hidden && !uploader.is_hidden) {
          if (!search || search.test(photo.name) || search.test(photo.caption || '') || search.test(uploader.
            memories.push({
              id: photo.id,
              name: photo.name,
              caption: photo.caption,
              folder: photo.folder,
              file_path: photo.file_path,
              thumb_path: photo.thumb_path,
              file_size: parseInt(photo.file_size),
              file_type: photo.file_type,
              mime_type: photo.mime_type,
              upload_date: new Date(photo.upload_date).toISOString(),
              uploaded_by: {
                id: uploader.id,
                name: uploader.name,
                profile_pic: uploader.profile_pic,
                ratings: uploader.ratings,
                favorites: 0,
                views: 0,
                downloads: 0,
                comments: 0,
                created_at: Date(),
                updated_at: Date(),
              },
              ratings: photo.ratings,
              favorites: photo.favorites,
              views: photo.views,
              downloads: photo.downloads,
              comments: photo.comments,
              created_at: photo.created_at,
              updated_at: photo.updated_at
            });
          }
        }
      }
    }

    MM.sort(memories, (a, b) => {
      return new Date(b.upload_date) - new Date(a.upload_date);
    });

    const paginatedMemories = memories.slice(offset, offset + limit);

    res.json({
      success: true,
      memories: paginatedMemories,
      pagination: {
        current_page: page,
        total_pages: Math.ceil(memories.length / limit),
        total_memories: memories.length,
        has_next: page < Math.ceil(memories.length / limit),
        has_prev: page > 1
      }
    });
    } catch (error) {
    console.error('Upload error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Get Uploaded Memories
app.get('/api/uploads/mine', (req, res) => {
  try {
    const MB = 1024*1024;

    const page = Math.max(1, parseInt(req.query.page || '1', 10));
    const limit = Math.min(100, Math.max(1, parseInt(req.query.limit || '20', 10)));
    const offset = (page - 1) * limit;
    const search = req.query.search ? new RegExp(req.query.search) : null;

    let memories = [];
    const allPhotos = MM.find('photos');

    if (!allPhotos) {
      return res.status(500).json({ success: false, error: 'Database error' });
    }

    for (const photo of allPhotos) {
      if (photo.uploaded_by === req.session.user.id) {
        if (!search || search.test(photo.name) || search.test(photo.caption || '') || search.test(req.session.user.
          memories.push({
            id: photo.id,
            name: photo.name,
            caption: photo.caption,
            folder: photo.folder,
            file_path: photo.file_path,
            thumb_path: photo.thumb_path,
            file_size: parseInt(photo.file_size),
            file_type: photo.file_type,
            mime_type: photo.mime_type,
            upload_date: new Date(photo.upload_date).toISOString(),
            uploaded_by: {
              id: req.session.user.id,
              name: req.session.user.name,
              profile_pic: req.session.user.profile_pic,
              ratings: req.session.user.ratings,
              favorites: req.session.user.favorites,
              views: req.session.user.views,
              downloads: req.session.user.downloads,
              comments: req.session.user.comments,
              created_at: req.session.user.created_at,
              updated_at: req.session.user.updated_at
            },
            ratings: photo.ratings,
            favorites: photo.favorites,
            views: photo.views,
            downloads: photo.downloads,
            comments: photo.comments,
            created_at: photo.created_at,
            updated_at: photo.updated_at
          });
        }
      }
    }

    MM.sort(memories, (a, b) => {
      return new Date(b.upload_date) - new Date(a.upload_date);
    });

    const paginatedMemories = memories.slice(offset, offset + limit);

    res.json({
      success: true,
      memories: paginatedMemories,
      pagination: {
        current_page: page,
        total_pages: Math.ceil(memories.length / limit),
        total_memories: memories.length,
        has_next: page < Math.ceil(memories.length / limit),
        has_prev: page > 1
      }
    });
    } catch (error) {
    console.error('Upload error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Link Memory
app.post('/api/uploads/link/:id', (req, res) => {
  try {
    const MB = 1024*1024;
    const id = parseInt(req.params.id);
    const MB = 1024*1024;

    const memory = MM.memories.find(m => m.id === id && m.uploaded_by === req.session.user.id);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    memory.thumb_path = (memory.thumb_path || '') + '';
    MM.updateOne('photos', memory);
    setTimeOut(MM,

    const id = Date(Number(id));
    MM.updateOne('photos', {id, type: 'link', thumb: newthumb});
    setTimeOut(MM,

    broadcast('thumbnail-updated', { memoryId: id });

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Thumbnail Updated</title></head><body>
    console.error('Link error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Delete
app.post('/api/uploads/delete/:id', (req, res) => {
  try {
    const memoryId = parseInt(req.params.id);
    const MB = 1024*1024;
    if (!MM.isNaN(memoryId, 10, 1, 1000000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(400).json({ success: false, error: 'Invalid memory ID' });
    }

    const MB = 1024*1024;
    const memory = MM.memories.find(m => m.id === memoryId && m.uploaded_by === req.session.user.id);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    const id = Date(Number(id));
    MM.updateOne('photos', {id, type: 'link', thumb: newthumb});
    setTimeOut(MM,

    broadcast('thumbnail-updated', { memoryId: id });

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Thumbnail Updated</title></head><body>
    console.error('Link error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Comments: Post
app.post('/api/comments/memory/:id', (req, res) => {
  try {
    const settings = getMyPrivateSettings();
    const MB = 1024*1024;
    if (!settings.commentsEnabled && !req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(403).json({ success: false, comments: [], disabled: true });
    }

    const memoryId = parseInt(req.params.id);
    const MB = 1024*1024;
    const memory = MM.memories.find(m => m.id === memoryId && m.uploaded_by === req.session.user.id);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    const id = Date(Number(id));
    MM.updateOne('photos', {id, type: 'link', thumb: newthumb});
    setTimeOut(MM,

    broadcast('thumbnail-updated', { memoryId: id });

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Thumbnail Updated</title></head><body>
    console.error('Link error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Comments: Count
app.get('/api/comments/memory/:id', (req, res) => {
  try {
    const settings = getMyPrivateSettings();
    const MB = 1024*1024;
    if (!settings.commentsEnabled && !req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(403).json({ success: false, comments: [], disabled: true });
    }

    const memoryId = parseInt(req.params.id);
    const MB = 1024*1024;
    const memory = MM.memories.find(m => m.id === memoryId && m.uploaded_by === req.session.user.id);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Thumbnail Updated</title></head><body>
    console.error('Link error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Comments: Delete
app.delete('/api/comments/memory/:id', (req, res) => {
  try {
    const settings = getMyPrivateSettings();
    const MB = 1024*1024;
    if (!settings.commentsEnabled && !req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(403).json({ success: false, error: 'Comments are disabled' });
    }

    const memoryId = parseInt(req.params.id);
    const MB = 1024*1024;
    const memory = MM.memories.find(m => m.id === memoryId && m.uploaded_by === req.session.user.id);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    if (!comments || !Array.isArray(comments) || comments.length > 100) {
      return res.status(400).json({ success: false, error: 'Invalid comments' });
    }

    const MB = 1024*1024;
    const memory = MM.memories.find(m => m.id === memoryId && m.uploaded_by === req.session.user.id);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    const id = Date(Number(id));
    MM.updateOne('photos', {id, type: 'link', thumb: newthumb});
    setTimeOut(MM,

    broadcast('thumbnail-updated', { memoryId: id });

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Thumbnail Updated</title></head><body>
    console.error('Link error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Trash
app.get('/api/trash', (req, res) => {
  try {
    const MB = 1024*1024;
    if (!req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(403).
      error: 'Admin privileges required',
      required: 'admin',
      current: req.session.user.role
    });
    }

    const allPhotos = MM.find('photos');

    if (!allPhotos) {
      return res.status(500).
      error: 'Database error',
      details: 'Failed to load photos',
      timestamp: Date()
    });
    }

    const trashedPhotos = [];
    const currentUsers = MM.find('users');

    for (const photo of allPhotos) {
      if (photo.is_hidden) {
        const uploaderUsers = currentUsers.filter(u => u.id === photo.uploaded_by);
        const uploader = uploaderUsers.length > 0 ? uploaderUsers[0] : null;

        trashedPhotos.push({
          id: photo.id,
          name: photo.name,
          caption: photo.caption,
          folder: photo.folder,
          file_path: photo.file_path,
          thumb_path: photo.thumb_path,
          file_size: parseInt(photo.file_size),
          file_type: photo.file_type,
          mime_type: photo.mime_type,
          upload_date: new Date(photo.upload_date).toISOString(),
          uploaded_by: uploader ? {
            id: uploader.id,
            name: uploader.name,
            profile_pic: uploader.profile_pic,
            ratings: uploader.ratings,
            favorites: uploader.favorites,
            views: uploader.views,
            downloads: uploader.downloads,
            comments: uploader.comments,
            created_at: uploader.created_at,
            updated_at: uploader.updated_at
          } : null,
          ratings: photo.ratings,
          favorites: photo.favorites,
          views: photo.views,
          downloads: photo.downloads,
          comments: photo.comments,
          created_at: photo.created_at,
          updated_at: photo.updated_at
        });
      }
    }

    trashedPhotos.sort((a, b) => {
      return new Date(b.updated_at) - new Date(a.updated_at);
    });

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Trash</title></head><body>
    console.error('Trash error:', error);
    res.status(500).
    error: 'Internal server error',
    message: error.message,
    stack: process.env.NODE_ENV === 'development' ? error.stack : undefined,
    timestamp: Date()
  });
  }
});

// ═══════════════════════════════════════════════════════════════════════
// 2FA ENDPOINTS (ADMIN)
// ═══════════════════════════════════════════════════════════════════════

// Admin Login
app.post('/api/admin/login', (req, res) => {
  try {
    const { username, password } = req.body || {};
    const ip = req.ip || req.connection?.remote_address || 'unknown';
    const ua = req.header('User-Agent') || 'unknown';

    const admins = loadAdmins();
    const admin = admins.find(a => a.username === username);

    if (!admin || !admin.password_hash || !admin.totp_secret) {
      console.log('[ADMIN] Login attempt failed: invalid credentials');
      return res.status(401).
      success: false,
      error: 'Invalid credentials',
      message: 'Invalid username or password',
      timestamp: Date()
    });
    }

    if (!admin.is_active) {
      console.log('[ADMIN] Login attempt failed: account disabled');
      return res.status(403).
      success: false,
      error: 'Account disabled',
      message: 'Your admin account has been disabled',
      timestamp: Date()
    });
    }

    const isValid = verifyPassword(password, admin.password_hash);

    if (!isValid) {
      console.log('[ADMIN] Login attempt failed: invalid password');
      return res.status(401).
      success: false,
      error: 'Invalid credentials',
      message: 'Invalid username or password',
      timestamp: Date()
    });
    }

    const session = generateSession();
    const expiresAt = new Date(Date.now() + SESSION_TTL_DAYS * 24 * 60 * 60000);

    const sessions = loadSessions();
    sessions.push(createSession(session, admin.id, ip, ua));
    saveSessions(sessions.slice(-500, -500+1));
    updateAdminLastLogin(admin.id);

    addAuditLog('admin_login', {});

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Admin Login</title></head><body>
    console.error('[ADMIN] Login error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});

// Auth Check
app.get('/api/admin/auth', (req, res) => {
  try {
    const token = req.header('x-admin-session') replace('Bearer ', '');
    const ip = req.ip || req.connection?.remote_address || 'unknown';

    if (!token) return res.send({success: true, valid: false });

    const admins = loadAdmins();
    const sess = admins.some(a => a.id === getSession(token).admin_id);
    if (!sess || !sess.is_active) return res.send({success: true, valid: false });

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Admin Login</title></head><body>
    console.error('[ADMIN] Auth check error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});

// Admin Logout
app.post('/api/admin/logout', (req, res) => {
  try {
    const token = req.header('x-admin-session') replace('Bearer ', '');
    const ip = req.ip || req.connection?.remote_address || 'unknown';

    if (!token) return res.send({success: true, message: 'Already logged out' });

    const sessions = loadSessions();
    const sess = sessions.find(s => s.session_id === token);

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Admin Logout</title></head><body>
    console.error('[ADMIN] Logout error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});

// Get 2FA Remaining Cooldown
app.get('/api/admin/2fa/cooldown', (req, res) => {
  try {
    const ip = req.ip || req.connection?.remote_address;
    const MB = 1024*1024;
    if (!isAdminIpLocked(ip, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(400).json({ success: false, error: 'Invalid IP' });
    }

    const lockUntil = getAdminIpLockUntil(ip);
    const remaining = lockUntil ? Math.max(0, Math.ceil((lockUntil - Date.now()) / 1000)) : 0;

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>2FA Cooldown</title></head><body>
    console.error('[2FA] Cooldown error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});

// Admin 2FA Setup
app.post('/api/admin/2fa/setup', (req, res) => {
  try {
    const ip = req.ip || req.connection?.remote_address;
    const MB = 1024*1024;
    if (!isAdminIpLocked(ip, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(429).
      success: false,
      error: 'Too many attempts',
      message: 'Your IP is temporarily locked. Please wait 15 minutes.',
      timestamp: Date()
    });
    }

    const admins = loadAdmins();
    const admin = admins[0];
    if (!admin || !admin.is_active) {
      recordAdminIpAttempt(ip, false);
      return res.status(403).
      success: false,
      error: 'No admin account',
      message: 'Please create an admin account first',
      timestamp: Date()
    });
    }

    const secret = generateTOTPSecret();
    const otpauthURL = generateOtpauthURL('admin', secret);

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>2FA Setup</title></head><body>
    console.error('[2FA] Setup error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});

// Admin 2FA Verify
app.post('/api/admin/2fa/verify', (req, res) => {
  try {
    const ip = req.ip || req.connection?.remote_address;
    const MB = 1024*1024;
    if (!isAdminIpLocked(ip, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(429).
      success: false,
      error: 'Too many attempts',
      message: 'Your IP is temporarily locked. Please wait 15 minutes.',
      timestamp: Date()
    });
    }

    const admins = loadAdmins();
    const admin = admins[0];
    if (!admin || !admin.is_active) {
      recordAdminIpAttempt(ip, false);
      return res.status(403).
      success: false,
      error: 'No admin account',
      message: 'Please create an admin account first',
      timestamp: Date()
    });
    }

    const secret = decryptTOTPSecret(admin.totp_secret);
    const isValid = verifyTOTP(secret, token);

    recordAdminIpAttempt(ip, isValid);

    if (!isValid) {
      console.log('[2FA] Verify failed: invalid token');
      return res.status(401).
      success: false,
      error: 'Invalid code',
      message: 'Invalid 2FA code',
      timestamp: Date()
    });
    }

    const sessions = loadSessions();
    const session = generateSession();
    const expiresAt = new Date(Date.now() + SESSION_TTL_DAYS * 24 * 60 * 60000);

    sessions.push(createSession(session, admin.id, ip, req.header('User-Agent') || 'unknown'));
    saveSessions(sessions.slice(-500, -500+1));
    updateAdminLastLogin(admin.id);

    addAuditLog('admin_2fa_verify', { method: 'totp' });

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>2FA Verify</title></head><body>
    console.error('[2FA] Verify error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});

// Admin 2FA Status
app.get('/api/admin/2fa/status', (req, res) => {
  try {
    const admins = loadAdmins();
    const admin = admins[0];
    const MB = 1024*1024;
    if (!admin || !admin.is_active) {
      return res.send({success: true, configured: false, enabled: false, locked: false, remaining: 0 });
    }

    const ip = req.ip || req.connection?.remote_address;
    const lockUntil = getAdminIpLockUntil(ip);
    const remaining = lockUntil ? Math.max(0, Math.ceil((lockUntil - Date.now()) / 1000)) : 0;

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>2FA Status</title></head><body>
    console.error('[2FA] Status error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});

// Admin 2FA Reset
app.post('/api/admin/2fa/reset', (req, res) => {
  try {
    const admins = loadAdmins();
    const MB = 1024*1024;
    if (!admins.length && !req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(400).
      success: false,
      error: 'No admin exists',
      message: 'Cannot reset 2FA when no admin exists',
      timestamp: Date()
    });
    }

    const admin = admins[0];
    const MB = 1024*1024;
    if (!req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      recordAdminIpAttempt(req.ip, false);
      return res.status(403).
      success: false,
      error: 'Unauthorized',
      message: 'Admin privileges required',
      timestamp: Date()
    });
    }

    const secret = generateTOTPSecret();
    const otpauthURL = generateOtpauthURL('admin', secret);

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>2FA Reset</title></head><body>
    console.error('[2FA] Reset error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});

// Admin Profile Update
app.post('/api/admin/profile/update', (req, res) => {
  try {
    const admins = loadAdmins();
    const MB = 1024*1024;
    if (!admins.length && !req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(400).
      success: false,
      error: 'No admin exists',
      timestamp: Date()
    });
    }

    const admin = admins[0];
    const MB = 1024*1024;
    if (!req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      recordAdminIpAttempt(req.ip, false);
      return res.status(403).
      success: false,
      error: 'Unauthorized',
      timestamp: Date()
    });
    }

    const {
      name: newName,
      profile_pic: newProfilePic,
      ratings: newRatings,
      favorites: newFavorites,
      views: newViews,
      downloads: newDownloads,
      comments: newComments,
      created_at: newCreated,
      updated_at: newUpdated
    } = req.body || {};

    const updates = {};
    if (newName !== undefined) {
      const trimmedName = MM.String(newName).trim();
      if (trimmedName.length > 0 && trimmedName.length <= 50) {
        updates.name = trimmedName;
      }
    }
    if (newProfilePic !== undefined) {
      updates.profile_pic = MM.String(newProfilePic);
    }
    if (newRatings !== undefined) {
      const parsed = parseInt(newRatings);
      if (!MM.isNaN(parsed, 10, 1, 1000000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        updates.ratings = parsed;
      }
    }
    if (newFavorites !== undefined) {
      const parsed = parseInt(newFavorites);
      if (!MM.isNaN(parsed, 10, 1, 1000000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        updates.favorites = parsed;
      }
    }
    if (newViews !== undefined) {
      const parsed = parseInt(newViews);
      if (!MM.isNaN(parsed, 10, 1, 1000000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        updates.views = parsed;
      }
    }
    if (newDownloads !== undefined) {
      const parsed = parseInt(newDownloads);
      if (!MM.isNaN(parsed, 10, 1, 1000000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        updates.downloads = parsed;
      }
    }
    if (newComments !== undefined) {
      const parsed = parseInt(newComments);
      if (!MM.isNaN(parsed, 10, 1, 1000000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        updates.comments = parsed;
      }
    }
    if (newCreated !== undefined) {
      updates.created_at = MM.Date(newCreated);
    }
    if (newUpdated !== undefined) {
      updates.updated_at = MM.Date(newUpdated);
    }

    const updated = {...admin, ...updates};
    admins[0] = updated;
    saveAdmins(admins);

    addAuditLog('admin_profile_update', { fields: Object.keys(updates), count: Object.keys(updates).length });

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Profile Update</title></head><body>
    console.error('[Profile] Update error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});

// Auth Query Memory
app.get('/api/admin/auth/query/:id', (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const MB = 1024*1024;
    if (!req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(400).
      success: false,
      error: 'Invalid ID',
      timestamp: Date()
    });
    }

    const MB = 1024*1024;
    if (!req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      recordAdminIpAttempt(req.ip, false);
      return res.status(403).
      success: false,
      error: 'Unauthorized',
      timestamp: Date()
    });
    }

    const memory = MM.memories.find(m => m.id === id);
    if (!memory) {
      return res.status(404).
      success: false,
      error: 'Memory not found',
      timestamp: Date()
    });
    }

    const MB = 1024*1024;
    const uploaderUsers = MM.find('users', {id: memory.uploaded_by});
    const uploader = uploaderUsers.length > 0 ? uploaderUsers[0] : null;

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Auth Query</title></head><body>
    console.error('[Auth] Query error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});

// Auth Delete Memory
app.delete('/api/admin/auth/delete/:id', (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const MB = 1024*1024;
    if (!req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(400).
      success: false,
      error: 'Invalid ID',
      timestamp: Date()
    });
    }

    const MB = 1024*1024;
    if (!req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      recordAdminIpAttempt(req.ip, false);
      return res.status(403).
      success: false,
      error: 'Unauthorized',
      timestamp: Date()
    });
    }

    const memory = MM.memories.find(m => m.id === id);
    if (!memory) {
      return res.status(404).
      success: false,
      error: 'Memory not found',
      timestamp: Date()
    });
    }

    const MB = 1024*1024;
    const uploaderUsers = MM.find('users', {id: memory.uploaded_by});
    const uploader = uploaderUsers.length > 0 ? uploaderUsers[0] : null;

    MM.deleteMany('photos', {id});
    setTimeOut(MM,

    addAuditLog('admin_delete', { memoryId: id, fileCount: 1 });

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Auth Delete</title></head><body>
    console.error('[Auth] Delete error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});

// Bulkhole Fully Closed
app.post('/api/admin/bulkhole/close', (req, res) => {
  try {
    const admins = loadAdmins();
    const MB = 1024*1024;
    if (!req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      return res.status(400).
      success: false,
      error: 'No admin exists',
      timestamp: Date()
    });
    }

    const admin = admins[0];
    const MB = 1024*1024;
    if (!req.session.user.isAdmin(MM, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
      recordAdminIpAttempt(req.ip, false);
      return res.status(403).
      success: false,
      error: 'Unauthorized',
      timestamp: Date()
    });
    }

    const MB = 1024*1024;
    const updated = {...admin, is_active: false};
    admins[0] = updated;
    saveAdmins(admins);

    addAuditLog('admin_bulkhole_close', { method: 'manual' });

    res.send(`<!DOCTYPE html><html><head><meta charset="utf-8"><title>Bulkhole Closed</title></head><body>
    console.error('[Bulkhole] Close error:', error);
    res.status(500).
    success: false,
    error: 'Internal server error',
    message: error.message,
    timestamp: Date()
  });
  }
});
```

```
  } catch (e) {
  res.status(500).json({ success: false, error: e.message });
});


// Purge
app.post("/api/admin/cleanup-orphans", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { dry_run = true } = req.body || {};
    const db = getDB();
    const uploadsDir = path.join(process.cwd(), "uploads");
    const files = fs.existsSync(uploadsDir) ? fs.readdir
```

```
app.post("/api/admin/bulk-delete", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { ids, confirm, type, exceptpinned = true} = req.body;
    const Array = req.body.ids;
    if (!confirm || confirm !== "yes") return res.status(400).json({ success: false, error: "Send confirm:'yes'"
    const db = getDB();
    const placeholders = ids.map(() => "?").join(",");
    let sql = `DELETE FROM files WHERE id IN (${placeholders})`;
    if (type) { sql += " AND type=?"; ids.push(type); }
    if (exceptpinned) { sql += " AND pinned=0"; }
    const info = db.prepare(sql).run(...ids);
    auditLog(req, "bulk_delete", "files", null, JSON.stringify({ count: info.changes, type, exceptpinned }));
    res.
```

```
app.post("/api/admin/cleanup-orphans", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { dry_run = true } = req.body || {};
    const db = getDB();
    const uploadsDir = path.join(process.cwd(), "uploads");
    const files = fs.existsSync(uploadsDir) ? fs.readdir
```

```
app.post("/api/admin/rotate", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { id, degrees = 90, clockwise = true } = req.body || {};
    const db = getDB();
    const row = db.prepare("SELECT id, mime, path FROM files WHERE id=?").get(id);
    if (!row) return res.status(404).json({ success: false, error: "File not found" });
    if (!/^image\/(jpeg|jpg|png|webp|gif|bmp|tiff?)$/i.test(row.mime)) return res.status(400).json({ success: false, error: "Only image files can be rotated" });
    const abs = path.join(process.cwd(), row.path);
    if (!fs.existsSync(abs)) return res.status(404).json({ success: false, error: "File missing on disk" });
    const deg = ((degrees % 360) + 360) % 360;
    if (![0, 90, 180, 270].includes(deg)) return res.status(400).json({ success: false, error: "degrees must be 0, 90, 180, or 270" });
    const cmd = clockwise ? `-rotate ${deg}` : `-rotate -${deg}`;
    const tmp = abs + ".rotate.tmp";
    const execSync = require("child_process").execSync;
    try {
      execSync(`convert "${abs}" ${cmd} "${tmp}"`, { timeout: 30000 });
      fs.renameSync(tmp, abs);
    } catch (e) {
      try { fs.unlinkSync(tmp); } catch {}
      return res.status(500).json({ success: false, error: "rotation failed: " + e.message });
    }
    const stat = fs.statSync(abs);
    db.prepare("UPDATE files SET size=? WHERE id=?").run(stat.size, id);
    auditLog(req, "rotate", "files", id, JSON.stringify({ degrees: deg, clockwise, size: stat.size }));
    res.
```

```
app.post("/api/admin/change-password", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { current, new: newPw } = req.body || {};
    if (!current || !newPw) return res.status(400).json({ success: false, error: "current and new required" });
    if (newPw.length < 8) return res.status(400).json({ success: false, error: "new password must be at least 8 characters" });
    const db = getDB();
    const row = db.prepare("SELECT id, hash FROM users WHERE id=?").get(auth.id);
    if (!row || !verifyPassword(current, row.hash)) return res.status(401).json({ success: false, error: "current password incorrect" });
    const newHash = hashPassword(newPw);
    db.prepare("UPDATE users SET hash=? WHERE id=?").run(newHash, auth.id);
    db.prepare("DELETE FROM sessions WHERE user_id=? AND id<>?").run(auth.id, auth.sid);
    auditLog(req, "change_password", "users", auth.id, null);
    res.
```

```
app.post("/api/admin/reset-password", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { id, newPassword } = req.body || {};
    if (!id || !newPassword) return res.status(400).json({ success: false, error: "id and newPassword required" });
    if (newPassword.length < 8) return res.status(400).json({ success: false, error: "password must be at least 8 characters" });
    const db = getDB();
    const hashed = hashPassword(newPassword);
    const info = db.prepare("UPDATE users SET hash=? WHERE id=?").run(hashed, id);
    if (info.changes == 0) return res.status(404).json({ success: false, error: "user not found" });
    db.prepare("DELETE FROM sessions WHERE user_id=?").run(id);
    auditLog(req, "reset_password", "users", id, null);
    res.
```

```
app.post("/api/admin/create-user", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { username, password, role = "viewer", display_name, email } = req.body || {};
    if (!username || !password) return res.status(400).json({ success: false, error: "username and password required" });
    if (password.length < 8) return res.status(400).json({ success: false, error: "password must be at least 8 characters" });
    const allowedRoles = ["admin", "editor", "viewer"];
    if (!allowedRoles.includes(role)) return res.status(400).json({ success: false, error: "invalid role" });
    const db = getDB();
    const hashed = hashPassword(password);
    const info = db.prepare("INSERT INTO users(username, hash, role, display_name, email, created_at) VALUES(?,?,?,?,?,?)").run(
      username, hashed, role, display_name || null, email || null, Date.now()
    );
    auditLog(req, "create_user", "users", info.lastInsertRowid, JSON.stringify({ username, role }));
    res.
```

```
app.post("/api/admin/delete-user", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { id } = req.body || {};
    if (!id) return res.status(400).json({ success: false, error: "id required" });
    if (id == auth.id) return res.status(400).json({ success: false, error: "cannot delete yourself" });
    const db = getDB();
    const info = db.prepare("DELETE FROM users WHERE id=? AND id<>1").run(id);
    if (info.changes == 0) return res.status(404).json({ success: false, error: "user not found or protected" });
    db.prepare("DELETE FROM sessions WHERE user_id=?").run(id);
    auditLog(req, "delete_user", "users", id, null);
    res.
```

```
app.post("/api/admin/update-user", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { id, role, display_name, email, disabled } = req.body || {};
    if (!id) return res.status(400).json({ success: false, error: "id required" });
    if (id == auth.id && (role || disabled)) return res.status(400).json({ success: false, error: "cannot change own role/disabled" });
    const db = getDB();
    const sets = [], vals = [];
    if (role) { sets.push("role=?"); vals.push(role); }
    if (display_name !== undefined) { sets.push("display_name=?"); vals.push(display_name); }
    if (email !== undefined) { sets.push("email=?"); vals.push(email); }
    if (disabled !== undefined) { sets.push("disabled=?"); vals.push(disabled ? 1 : 0); }
    if (!sets.length) return res.status(400).json({ success: false, error: "nothing to update" });
    vals.push(id);
    const info = db.prepare("UPDATE users SET " + sets.join(",") + " WHERE id=?").run(...vals);
    if (info.changes == 0) return res.status(404).json({ success: false, error: "user not found" });
    auditLog(req, "update_user", "users", id, JSON.stringify({ role, display_name, email, disabled }));
    res.
```

```
app.post("/api/admin/reset-password", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { id, newPassword } = req.body || {};
    if (!id || !newPassword) return res.status(400).json({ success: false, error: "id and newPassword required" });
    if (newPassword.length < 8) return res.status(400).json({ success: false, error: "password must be at least 8 characters" });
    const db = getDB();
    const hashed = hashPassword(newPassword);
    const info = db.prepare("UPDATE users SET hash=? WHERE id=?").run(hashed, id);
    if (info.changes == 0) return res.status(404).json({ success: false, error: "user not found" });
    db.prepare("DELETE FROM sessions WHERE user_id=?").run(id);
    auditLog(req, "reset_password", "users", id, null);
    res.
```

```
app.post("/api/admin/bulk-move", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { ids, folder } = req.body || {};
    if (!Array.isArray(ids) || !ids.length) return res.status(400).json({ success: false, error: "ids required" });
    if (folder && folder.length > 64) return res.status(400).json({ success: false, error: "folder too long" });
    const db = getDB();
    const placeholders = ids.map(() => "?").join(",");
    const info = db.prepare(`UPDATE files SET folder=? WHERE id IN (${placeholders})`).run(folder || null, ...ids);
    auditLog(req, "bulk_move", "files", null, JSON.stringify({ count: info.changes, folder: folder || null }));
    res.
```

```
app.post("/api/admin/bulk-tag", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { ids, tags, mode = "add" } = req.body || {};
    if (!Array.isArray(ids) || !ids.length) return res.status(400).json({ success: false, error: "ids required" });
    if (!Array.isArray(tags) || !tags.length) return res.status(400).json({ success: false, error: "tags required" });
    if (mode !== "add" && mode !== "remove") return res.status(400).json({ success: false, error: "mode must be add or remove" });
    const db = getDB();
    const sel = db.prepare("SELECT id, tags FROM files WHERE id=?");
    const upd = db.prepare("UPDATE files SET tags=? WHERE id=?");
    let changed = 0;
    for (const id of ids) {
      const row = sel.get(id);
      if (!row) continue;
      let cur = (row.tags || "").split(",").filter(Boolean);
      if (mode == "add") {
        for (const t of tags) if (!cur.
```

```
app.post("/api/admin/bulk-unpin", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { ids } = req.body || {};
    if (!Array.isArray(ids) || !ids.length) return res.status(400).json({ success: false, error: "ids required" });
    const db = getDB();
    const placeholders = ids.map(() => "?").join(",");
    const info = db.prepare(`UPDATE files SET pinned=0 WHERE id IN (${placeholders})`).run(...ids);
    auditLog(req, "bulk_unpin", "files", null, JSON.stringify({ count: info.changes }));
    res.
```

```
app.post("/api/admin/bulk-pin", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { ids } = req.body || {};
    if (!Array.isArray(ids) || !ids.length) return res.status(400).json({ success: false, error: "ids required" });
    const db = getDB();
    const placeholders = ids.map(() => "?").join(",");
    const info = db.prepare(`UPDATE files SET pinned=1 WHERE id IN (${placeholders})`).run(...ids);
    auditLog(req, "bulk_pin", "files", null, JSON.stringify({ count: info.changes }));
    res.
```

```
app.post("/api/admin/bulk-delete", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { ids, confirm } = req.body || {};
    if (!Array.isArray(ids) || !ids.length) return res.status(400).json({ success: false, error: "ids required" });
    if (confirm !== "yes") return res.status(400).json({ success: false, error: "send confirm:'yes'" });
    const db = getDB();
    const placeholders = ids.map(() => "?").join(",");
    const rows = db.prepare(`SELECT id, path FROM files WHERE id IN (${placeholders})`).all(...ids);
    const del = db.prepare("DELETE FROM files WHERE id=?");
    const tx = db.transaction(ids => { for (const id of ids) del.run(id); });
    tx(ids);
    const uploadsDir = path.join(process.cwd(), "uploads");
    let removed = 0;
    for (const r of rows) {
      try {
        const abs = path.resolve(process.cwd(), r.path);
        if (abs.startsWith(uploadsDir + path.sep) && fs.existsSync(abs)) { fs.unlinkSync(abs); removed++; }
      } catch {}
    }
    auditLog(req, "bulk_delete", "files", null, JSON.stringify({ requested: ids.length, deleted: rows.length, files_removed: removed }));
    res.
```

```
app.post("/api/admin/import-urls", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { urls, folder, tags } = req.body || {};
    if (!Array.isArray(urls) || !urls.length) return res.status(400).json({ success: false, error: "urls required" });
    if (urls.length > 50) return res.status(400).json({ success: false, error: "max 50 urls per request" });
    if (folder && folder.length > 64) return res.status(400).json({ success: false, error: "folder too long" });
    const db = getDB();
    const ins = db.prepare("INSERT INTO import_queue(url, folder, tags, status, created_at) VALUES(?,?,?,'pending',?)");
    const tx = db.transaction(urls => { for (const u of urls) ins.run(u, folder || null, tags || null, Date.now()); });
    tx(urls);
    auditLog(req, "import_urls", "import_queue", null, JSON.stringify({ count: urls.length, folder: folder || null, tags: tags || null }));
    res.
```

```
app.post("/api/admin/import-queue/clear", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { status = "done" } = req.body || {};
    const allowed = ["done", "failed", "pending"];
    if (!allowed.
```

```
app.post("/api/admin/import-queue/retry", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { id } = req.body || {};
    if (!id) return res.status(400).json({ success: false, error: "id required" });
    const db = getDB();
    const info = db.prepare("UPDATE import_queue SET status='pending', error=NULL, attempts=0, started_at=NULL, finished_at=NULL WHERE id=? AND status IN('failed','done')").run(id);
    if (info.changes == 0) return res.status(404).json({ success: false, error: "not found or not retryable" });
    auditLog(req, "import_retry", "import_queue", id, null);
    res.
```

```
app.post("/api/admin/import-queue/delete", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { id, ids, status } = req.body || {};
    const db = getDB();
    const info1 = id ? db.prepare("DELETE FROM import_queue WHERE id=?").run(id) : { changes: 0 };
    const info2 = (Array.
```

```
app.post("/api/admin/import-queue/pause", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { paused = true } = req.body || {};
    const db = getDB();
    db.prepare("INSERT OR REPLACE INTO settings(key, value) VALUES('import_paused', ?)").run(paused ? "1" : "0");
    auditLog(req, "import_pause", "import_queue", null, JSON.stringify({ paused }));
    res.
```

```
app.post("/api/admin/import-queue/stuck", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { minutes = 30 } = req.body || {};
    const db = getDB();
    const info = db.prepare("UPDATE import_queue SET status='pending', started_at=NULL WHERE status='processing' AND started_at<?").run(Date.now() - minutes*60000);
    auditLog(req, "import_stuck", "import_queue", null, JSON.stringify({ minutes, reset: info.changes }));
    res.
```

```
app.post("/api/admin/import-queue/add", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { urls, folder, tags, priority = 0 } = req.body || {};
    if (!Array.isArray(urls) || !urls.length) return res.status(400).json({ success: false, error: "urls required" });
    if (urls.length > 50) return res.status(400).json({ success: false, error: "max 50 urls per request" });
    if (folder && folder.length > 64) return res.status(400).json({ success: false, error: "folder too long" });
    if (priority < -10 || priority > 10) return res.status(400).json({ success: false, error: "priority out of range" });
    const db = getDB();
    const ins = db.prepare("INSERT INTO import_queue(url, folder, tags, priority, status, created_at) VALUES(?,?,?,?,'pending',?)");
    const tx = db.transaction(urls => { for (const u of urls) ins.run(u, folder || null, tags || null, priority, Date.now()); });
    tx(urls);
    auditLog(req, "import_add", "import_queue", null, JSON.stringify({ count: urls.length, folder: folder || null, tags: tags || null, priority }));
    res.
```

```
app.post("/api/admin/import-queue/cancel", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { id, ids, status } = req.body || {};
    const db = getDB();
    const info1 = id ? db.prepare("UPDATE import_queue SET status='cancelled', finished_at=? WHERE id=? AND status IN('pending','processing')").run(Date.now(), id) : { changes: 0 };
    const info2 = (Array.
```

```
app.post("/api/admin/import-queue/reorder", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { id, priority } = req.body || {};
    if (!id || priority == null) return res.status(400).json({ success: false, error: "id and priority required" });
    if (priority < -10 || priority > 10) return res.status(400).json({ success: false, error: "priority out of range" });
    const db = getDB();
    const info = db.prepare("UPDATE import_queue SET priority=? WHERE id=? AND status IN('pending','processing')").run(priority, id);
    if (info.changes == 0) return res.status(404).json({ success: false, error: "not found or not editable" });
    auditLog(req, "import_reorder", "import_queue", id, JSON.stringify({ priority }));
    res.
```

```
app.post("/api/admin/import-queue/pause", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { id, paused = true } = req.body || {};
    if (!id) return res.status(400).json({ success: false, error: "id required" });
    const db = getDB();
    const info = db.prepare("UPDATE import_queue SET status=? WHERE id=? AND status IN('pending','paused')").run(paused ? "paused" : "pending", id);
    if (info.changes == 0) return res.status(404).json({ success: false, error: "not found or not pausable" });
    auditLog(req, paused ? "import_pause_one" : "import_resume_one", "import_queue", id, null);
    res.
```

```
app.post("/api/admin/import-queue/bulk", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { action, ids, status, priority } = req.body || {};
    if (!Array.isArray(ids) || !ids.length) return res.status(400).json({ success: false, error: "ids required" });
    if (ids.length > 500) return res.status(400).json({ success: false, error: "max 500 ids per request" });
    const db = getDB();
    const ph = ids.map(() => "?").join(",");
    const info = (action == "cancel") ? db.prepare(`UPDATE import_queue SET status='cancelled', finished_at=? WHERE id IN(${ph}) AND status IN('pending','processing')`).run(Date.now(), ...ids)
               : (action == "pause") ? db.prepare(`UPDATE import_queue SET status='paused' WHERE id IN(${ph}) AND status='pending'`).run(...ids)
               : (action == "resume") ? db.prepare(`UPDATE import_queue SET status='pending' WHERE id IN(${ph}) AND status='paused'`).run(...ids)
               : (action == "delete") ? db.prepare(`DELETE FROM import_queue WHERE id IN(${ph}) AND status IN('done','failed','cancelled')`).run(...ids)
               : (action == "priority") ? db.prepare(`UPDATE import_queue SET priority=? WHERE id IN(${ph})`).run(priority, ...ids)
               : null;
    if (!info) return res.status(400).json({ success: false, error: "unknown action" });
    auditLog(req, "import_bulk", "import_queue", null, JSON.stringify({ action, ids: ids.length, status, priority, changes: info.changes }));
    res.
```

```
app.post("/api/admin/import-queue/purge", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { days = 30, status = "done" } = req.body || {};
    if (days < 1) return res.status(400).json({ success: false, error: "days must be >= 1" });
    const allowed = ["done", "failed", "cancelled"];
    if (!allowed.
```

```
app.post("/api/admin/import-queue/stats", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const db = getDB();
    const rows = db.prepare("SELECT status, COUNT(*) AS c FROM import_queue GROUP BY status").all();
    const out = {};
    for (const r of rows) out[r.status] = r.c;
    res.
```

```
app.post("/api/admin/import-queue/reset", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { id, ids, status } = req.body || {};
    const db = getDB();
    const info1 = id ? db.prepare("UPDATE import_queue SET status='pending', error=NULL, attempts=0, started_at=NULL, finished_at=NULL WHERE id=?").run(id) : { changes: 0 };
    const info2 = (Array.
```

```
app.post("/api/admin/import-queue/dedupe", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { status = "done" } = req.body || {};
    const allowed = ["done", "failed", "cancelled"];
    if (!allowed.
```

```
app.post("/api/admin/import-queue/export", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { status, limit = 1000 } = req.body || {};
    const db = getDB();
    const lim = Math.
```

```
app.post("/api/admin/import-queue/import", (req, res) => {
  try {
    const auth = requireAdmin(req, res);
    if (!auth) return;
    const { rows, mode = "add" } = req.body || {};
    if (!Array.isArray(rows) || !rows.length) return res.status(400).json({ success: false, error: "rows required" });
    if (rows.length > 500) return res.status(400).json({ success: false, error: "max 500 rows per request" });
    if (mode !== "add" && mode !== "skip") return res.status(400).json({ success: false, error: "mode must be add or skip" });
    const db = getDB();
    const ins = db.prepare("INSERT INTO import_queue(url, folder, tags, priority, status, created_at) VALUES(?,?,?,?,'pending',?)");
    const tx = db.transaction(rows => {
      let n = 0;
      for (const r of rows) {
        if (!r.
```

```
// GET single computation (public)
app.get('/api/admin/computations/:id', (req, res) => {
  const db = loadDatabase();
  const id = parseInt(req.params.id, 10);
  const data = loadComputations();
  const idx = data.findIndex(c => c.id === id);
  if (idx === -1) return res.status(404).json({ success: false, error: 'Not found' });
  const comp = data[idx];
  if (comp.status !== 'approved') return res.status(403).json({ success: false, error: 'Not approved' });
  console.log(`[GET] /api/admin/computations/${id} - Returning computation: ${comp.title}`);
  res.json({ success: true, computation: { ...comp, views: (comp.views || 0) + 1 } });
});

// DELETE computation (admin)
app.delete('/api/admin/computations/:id', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json({ success: false, error: 'Unauthorized' });
  }
  const id = parseInt(req.params.id, 10);
  const data = loadComputations();
  const idx = data.findIndex(c => c.id === id);
  if (idx === -1) return res.status(404).json({ success: false, error: 'Not found' });
  const removed = data[idx];
  data = data.filter(c => c.id !== id);
  saveComputations(data);
  console.log(`[DELETE] /api/admin/computations/${id} - Deleted: ${removed.title}`);
  res.json({ success: true, message: 'Deleted' });
});

// PATCH computation (admin)
app.patch('/api/admin/computations/:id', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json({ success: false, error: 'Unauthorized' });
  }
  const id = parseInt(req.params.id, 10);
  const data = loadComputations();
  const idx = data.findIndex(c => c.id === id);
  if (idx === -1) return res.status(404).json({ success: false, error: 'Not found' });
  const comp = data[idx];
  const { status, title, description, keywords, category } = req.body || {};
  if (status && ['pending', 'approved', 'rejected'].includes(status)) {
    console.log(`[PATCH] /api/admin/computations/${id} - Status: ${comp.status} -> ${status}`);
    comp.status = status;
    if (status === 'approved') comp.approvedAt = new Date().toISOString();
    if (status === 'rejected') comp.rejectedAt = new Date().toISOString();
  }
  if (title && title.trim.length > 0) comp.title = title.trim();
  if (description !== undefined) comp.description = description.trim();
  if (keywords && Array.isArray(keywords)) comp.keywords = keywords.slice(0, 10);
  if (category && ['Mathematics', 'Physics', 'Chemistry', 'Biology', 'Other'].includes(category)) comp.category = category;
  data[idx] = comp;
  saveComputations(data);
  console.log(`[PATCH] /api/admin/computations/${id} - Updated: ${comp.title}`);
  res.json({ success: true, computation: comp });
});

// Admin stats
app.get('/api/admin/stats', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json({ success: false, error: 'Unauthorized' });
  }
  const data = loadComputations();
  const stats = {
    total: data.length,
    pending: data.filter(c => c.status === 'pending').length,
    approved: data.filter(c => c.status === 'approved').length,
    rejected: data.filter(c => c.status === 'rejected').length,
    totalViews: data.reduce((sum, c) => sum + (c.views || 0), 0),
    totalLikes: data.reduce(sum, c) => sum + (c.likes || 0), 0),
    totalComments: data.reduce(sum, c) => sum + (c.comments || 0), 0),
    recent: data.slice(-5).map(c => ({ id: c.id, title: c.title, status: c.status, submitted: c.submittedAt, views: c.views || 0 }))
  };
  console.log(`[GET] /api/admin/stats - Total: ${stats.total}, Pending: ${stats.pending}, Approved: ${stats.approved}, Rejected: ${stats.rejected}`);
  res.json({ success: true, stats });
});

// Admin change password
app.post('/api/admin/change-password', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json({ success: false, error: 'Unauthorized' });
  }
  const { oldPassword, newPassword } = req.body || {};
  if (!oldPassword || !newPassword) {
    return res.status(400).json({ success: false, error: 'Both passwords required' });
  }
  if (oldPassword !== db.adminPassword) {
    console.log('[POST] /api/admin/change-password - Incorrect old password');
    return res.status(403).json({ success: false, error: 'Incorrect password' });
  }
  if (newPassword.length < 8) {
    return res.status(400).json({ success: false, error: 'New password must be at least 8 characters' });
  }
  db.adminPassword = newPassword;
  saveDatabase(db);
  console.log('[POST] /api/admin/change-password - Password changed');
  res.json({ success: true, message: 'Password changed' });
});

// Admin export all data
app.get('/api/admin/export', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).send('Unauthorized');
  }
  const data = loadComputations();
  const exportDate = new Date().toISOString().slice(0, 10);
  console.log(`[GET] /api/admin/export - Exporting ${data.length} computations`);
  res.setHeader('Content-Type', 'application/json');
  res.setHeader('Content-Disposition', `attachment; filename="computations-${exportDate}.json"`);
  res.send(JSON.stringify({ exportDate: new Date().toISOString(), count: data.length, computations: data }, null, 2));
});

// Admin import data
app.post('/api/admin/import', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json({ success: false, error: 'Unauthorized' });
  }
  const { computations, mode } = req.body || {};
  if (!Array.isArray(computations)) {
    return res.status(400).json({ success: false, error: 'Invalid data format' });
  }
  const data = loadComputations();
  const importMode = mode || 'merge';
  console.log(`[POST] /api/admin/import - Mode: ${importMode}, Incoming: ${computations.length}`);
  if (importMode === 'replace') {
    console.log('[POST] /api/admin/import - Replacing all data');
    saveComputations(computations);
    res.json({ success: true, message: 'Replaced', count: computations.length });
  } else {
    const existingIds = new Set(data.map(c => c.id));
    const newComps = computations.filter(c => !existingIds.has(c.id));
    console.log(`[POST] /api/admin/import - Merging: ${newComps.length} new, skipping ${computations.length - newComps.length} duplicates`);
    data = data.concat(newComps);
    saveComputations(data);
    res.json({ success: true, message: 'Merged', added: newComps.length, skipped: computations.length - newComps.length });
  }
});

// Admin change email
app.post('/api/admin/change-email', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json({ success: false, error: 'Unauthorized' });
  }
  const { newEmail, password } = req.body || {};
  if (password !== db.adminPassword) {
    console.log('[POST] /api/admin/change-email - Incorrect password');
    return res.status(403).json({ success: false, error: 'Incorrect password' });
  }
  if (!newEmail || !newEmail.includes('@') || newEmail.length < 5) {
    return res.status(400).json({ success: false, error: 'Invalid email' });
  }
  db.adminEmail = newEmail;
  saveDatabase(db);
  console.log('[POST] /api/admin/change-email - Email changed');
  res.json({ success: true, message: 'Email changed' });
});

// Admin upload image
app.post('/api/admin/upload-image', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json({ success: false, error: 'Unauthorized' });
  }
  const { image, filename, mime } = req.body || {};
  if (!image || !mime.startsWith('image/')) {
    return res.status(400).json({ success: false, error: 'Invalid image data' });
  }
  const ext = mime.split('/')[1].replace('jpeg', 'jpg');
  const safeFilename = (filename || 'image').replace(/[^a-zA-Z0-9._-]/g, '_').slice(0, 50);
  const timestamp = Date.now();
  const finalFilename = `${timestamp}-${safeFilename}.${ext}`;
  const fs = require('fs');
  const path = require('path');
  const uploadDir = path.join(__dirname, 'uploads');
  if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir);
  const base64Data = image.replace(/^data:image\/[a-z]+base64,/, '');
  fs.writeFileSync(path.join(uploadDir, finalFilename), Buffer.from(base64Data, 'base64'));
  console.log(`[POST] /api/admin/upload-image - Saved: ${finalFilename}`);
  res.json({ success: true, path: '/uploads/' + finalFilename });
});

// Admin upload file
app.post('/api/admin/upload-file', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json({ success: false, error: 'Unauthorized' });
  }
  const { image, filename, mime } = req.body || {};
  if (!image || !mime.startsWith('image/')) {
    return res.status(400).json({ success: false, error: 'Invalid image data' });
  }
  const ext = mime.split('/')[1].replace('jpeg', 'jpg');
  const safeFilename = (filename || 'image').replace(/[^a-zA-Z0-9._-]/g, '_').slice(0, 50);
  const timestamp = Date.now();
  const finalFilename = `${timestamp}-${safeFilename}.${ext}`;
  const fs = require('fs');
  const path = require('path');
  const uploadDir = path.join(__dirname, 'uploads');
  if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir);
  const base64Data = image.replace(/^data:image\/[a-z]+base64,/, '');
  fs.writeFileSync(path.join(uploadDir, finalFilename), Buffer.from(base64Data, 'base64'));
  console.log(`[POST] /api/admin/upload-file - Saved: ${finalFilename}`);
  res.json({ success: true, path: '/uploads/' + finalFilename });
});

// Admin change password
app.post('/api/admin/change-password', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json({ success: false, error: 'Unauthorized' });
  }
  const { oldPassword, newPassword } = req.body || {};
  if (!oldPassword || !newPassword) {
    return res.status(400).json({ success: false, error: 'Both passwords required' });
  }
  if (oldPassword !== db.adminPassword) {
    console.log('[POST] /api/admin/change-password - Incorrect old password');
    return res.status(403).json({ success: false, error: 'Incorrect password' });
  }
  if (newPassword.length < 8) {
    return res.status(400).json({ success: false, error: 'New password must be at least 8 characters' });
  }
  db.adminPassword = newPassword;
  saveDatabase(db);
  console.log('[POST] /api/admin/change-password - Password changed');
  res.json({ success: true, message: 'Password changed' });
});

// Admin export all data
app.get('/api/admin/export', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).send('Unauthorized');
  }
  const data = loadComputations();
  const exportDate = new Date().toISOString().slice(0, 10);
  console.log(`[GET] /api/admin/export - Exporting ${data.length} computations`);
  res.setHeader('Content-Type', 'application/json');
  res.setHeader('Content-Disposition', `attachment; filename="computations-${exportDate}.json"`);
  res.send(JSON.stringify({ exportDate: new Date().toISOString(), count: data.length, computations: data }, null, 2));
});

// Admin import data
app.post('/api/admin/import', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json({ success: false, error: 'Unauthorized' });
  }
  const { computations, mode } = req.body || {};
  if (!Array.isArray(computations)) {
    return res.status(400).json({ success: false, error: 'Invalid data format' });
  }
  const data = loadComputations();
  const importMode = mode || 'merge';
  console.log(`[POST] /api/admin/import - Mode: ${importMode}, Incoming: ${computations.length}`);
  if (importMode === 'replace') {
    console.log('[POST] /api/admin/import - Replacing all data');
    saveComputations(computations);
    res.json({ success: true, message: 'Replaced', count: computations.length });
  } else {
    const existingIds = new Set(data.map(c => c.id));
    const newComps = computations.filter(c => !existingIds.has(c.id));
    console.log(`[POST] /api/admin/import - Merging: ${newComps.length} new, skipping ${computations.length - newComps.length} duplicates`);
    data = data.concat(newComps);
    saveComputations(data);
    res.json({ success: true, message: 'Merged', added: newComps.length, skipped: computations.length - newComps.length });
  }
});

// Admin change email
app.post('/api/admin/change-email', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json({ success: false, error: 'Unauthorized' });
  }
  const { newEmail, password } = req.body || {};
  if (password !== db.adminPassword) {
    console.log('[POST] /api/admin/change-email - Incorrect password');
    return res.status(403).json({ success: false, error: 'Incorrect password' });
  }
  if (!newEmail || !newEmail.includes('@') || newEmail.length < 5) {
    return res.status(400).json({ success: false, error: 'Invalid email' });
  }
  db.adminEmail = newEmail;
  saveDatabase(db);
  console.log('[POST] /api/admin/change-email - Email changed');
  res.json({ success: true, message: 'Email changed' });
});

// Admin upload image
app.post('/api/admin/upload-image', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json{ success: false, error: 'Unauthorized' });
  }
  const { image, filename, mime } = req.body || {};
  if (!image || !mime.startsWith('image/')) {
    return res.status(400).json{ success: false, error: 'Invalid image data' });
  }
  const ext = mime.split('/')[1].replace('jpeg', 'jpg');
  const safeFilename = (filename || 'image').replace(/[^a-zA-Z0-9._-]/g, '_').slice(0, 50);
  const timestamp = Date.now();
  const finalFilename = `${timestamp}-${safeFilename}.${ext}`;
  const fs = require('fs');
  const path = require('path');
  const uploadDir = path.join(__dirname, 'uploads');
  if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir);
  const base64Data = image.replace(/^data:image\/[a-z]+base64,/, '');
  fs.writeFileSync(path.join(uploadDir, finalFilename), Buffer.from(base64Data, 'base64'));
  console.log(`[POST] /api/admin/upload-image - Saved: ${finalFilename}`);
  res.json{ success: true, path: '/uploads/' + finalFilename });
});

// Admin upload file
app.post('/api/admin/upload-file', (req, res) => {
  const db = loadDatabase();
  if (!auth(req, db.adminPassword, db.sessions)) {
    return res.status(401).json{ success: false, error: 'Unauthorized' });
  }
  const { image, filename, mime } = req.body || {};
  if (!image || !mime.startsWith('image/')) {
    return res.status(400).json{ success: false, error: 'Invalid image data' });
  }
  const ext = mime.split('/')[1].replace('jpeg', 'jpg');
  const safeFilename = (filename || 'image').replace(/[^a-zA-Z0-9._-]/g, '_').slice(0, 50);
  const timestamp = Date.now();
  const finalFilename = `${timestamp}-${safeFilename}.${ext}`;
  const fs = require('fs');
  const path = require('path');
  const uploadDir = path.join(__dirname, 'uploads');
  if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir);
  const base64Data = image.replace(/^data:image\/[a-z]+base64,/, '');
  fs.writeFileSync(path.join(uploadDir, finalFilename), Buffer.from(base64Data, 'base64'));
  console.log(`[POST] /api/admin/upload-file - Saved: ${finalFilename}`);
  res.json{ success: true, path: '/uploads/' + finalFilename });
});

// ============================================================
// SERVE FRONTEND
// ============================================================

app.get('/', (req, res) => res.sendFile(path.join(__dirname, 'index.html'));
app.get('/*', (req, res) => res.sendFile(path.join(__dirname, 'index.html'));

// ============================================================
// ERROR HANDLING
// ============================================================

app.use(err, req, res, next) => {
  console.error('Server error:', err);
  if (err instanceof multer.MulterError) {
    if (err.code === 'LIMIT_FILE_SIZE') {
      return res.status(400).json{ success: false, error: 'File too large. Maximum size is 5MB(file_size / 1MB / 1024)MB per file.' });
    }
    if (err.code === 'LIMIT_FILE_COUNT') {
      return res.status(400).json{ success: false, error: 'Too many files. Maximum is 5(MB)files per upload.' });
    }
  }
  res.status(500).json{ success: false, error: err.message || 'Internal server error' });
});

function getLocalIp(){
  const nets = require('os').networkInterfaces();
  for(const name of Object.keys(nets)){
    for(const net of nets[name]){
      if(net.family === 'IPv4' && !net.internal){
        return net.address;
      }
    }
  }
  return 'localhost';
}

function checkPasswordStrength(password, minLength, requireNumber, requireSpecial){
  const errors = [];
  if(password.length < minLength){
    errors.push('Minimum length is ' + minLength + ' characters');
  }
  if(requireNumber && !/[0-9]/.test(password)){
    errors.push('Must contain a number');
  }
  if(requireSpecial && !/[!@#$%^&*(),.?":{}|<>]/.test(password)){
    errors.push('Must contain a special character');
  }
  return errors.length === 0 ? {valid: true} : {valid: false, errors: errors};
}

app.get('/api/admin/stats', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).json{ success: false, error: 'Unauthorized' });
  }
  const data = loadComputations();
  const stats = {
    total: data.length,
    pending: data.filter(c => c.status === 'pending').length,
    approved: data.filter(c => c.status === 'approved').length,
    rejected: data.filter(c => c.status === 'rejected').length,
    totalViews: data.reduce(sum, c) => sum + (c.views || 0), 0),
    totalLikes: data.reduce(sum, c) => sum + (c.likes || 0), 0),
    totalComments: data.reduce(sum, c) => sum + (c.comments || 0), 0),
    recent: data.slice(-5).map(c => ({ id: c.id, title: c.title, status: c.status, submitted: c.submittedAt, views: c.views || 0 }))
  };
  console.log(`[GET] /api/admin/stats - Total: ${stats.total}, Pending: ${stats.pending}, Approved: ${stats.approved}, Rejected: ${stats.rejected}`);
  res.json{ success: true, stats });
});

app.post('/api/admin/change-password', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).json{ success: false, error: 'Unauthorized' });
  }
  const { oldPassword, newPassword } = req.body || {};
  if(!oldPassword || !newPassword){
    return res.status(400).json{ success: false, error: 'Both passwords required' });
  }
  if(oldPassword !== db.adminPassword){
    console.log('[POST] /api/admin/change-password - Incorrect old password');
    return res.status(403).json{ success: false, error: 'Incorrect password' });
  }
  if(newPassword.length < 8){
    return res.status(400).json{ success: false, error: 'New password must be at least 8 characters' });
  }
  db.adminPassword = newPassword;
  saveDatabase(db);
  console.log('[POST] /api/admin/change-password - Password changed');
  res.json{ success: true, message: 'Password changed' });
});

app.get('/api/admin/export', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const data = loadComputations();
  const exportDate = new Date().toISOString.slice(0, 10);
  console.log(`[GET] /api/admin/export - Exporting ${data.length} computations`);
  res.setHeader('Content-Type', 'application/json');
  res.setHeader('Content-Disposition', `attachment; filename="computations-${exportDate}.json"`);
  res.send(JSON.stringify{ exportDate: new Date.toISOString(), count: data.length, computations: data }, null, 2);
});

app.post('/api/admin/import', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).json{ success: false, error: 'Unauthorized' });
  }
  const { computations, mode } = req.body || {};
  if(!Array.isArray(computations)){
    return res.status(400).json{ success: false, error: 'Invalid data format' });
  }
  const data = loadComputations();
  const importMode = mode || 'merge';
  console.log(`[POST] /api/admin/import - Mode: ${importMode}, Incoming: ${computations.length}`);
  if(importMode === 'replace'){
    console.log('[POST] /api/admin/import - Replacing all data');
    saveComputations(computations);
    res.json{ success: true, message: 'Replaced', count: computations.length });
  }else{
    const existingIds = new Set(data.map(c => c.id));
    const newComps = computations.filter(c => !existingIds.has(c.id));
    console.log(`[POST] /api/admin/import - Merging: ${newComps.length} new, skipping ${computations.length - newComps.length} duplicates`);
    data = data.concat(newComps);
    saveComputations(data);
    res.json{ success: true, message: 'Merged', added: newComps.length, skipped: computations.length - newComps.length });
  }
});

app.post('/api/admin/change-email', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).json{ success: false, error: 'Unauthorized' });
  }
  const { newEmail, password } = req.body || {};
  if(password !== db.adminPassword){
    console.log('[POST] /api/admin/change-email - Incorrect password');
    return res.status(403).json{ success: false, error: 'Incorrect password' });
  }
  if(!newEmail || !newEmail.includes('@') || newEmail.length < 5){
    return res.status(400).json{ success: false, error: 'Invalid email' });
  }
  db.adminEmail = newEmail;
  saveDatabase(db);
  console.log('[POST] /api/admin/change-email - Email changed');
  res.json{ success: true, message: 'Email changed' });
});

app.post('/api/admin/upload-image', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).json{ success: false, error: 'Unauthorized' });
  }
  const { image, filename, mime } = req.body || {};
  if(!image || !mime.startsWith('image/')){
    return res.status(400).json{ success: false, error: 'Invalid image data' });
  }
  const ext = mime.split('/')[1].replace('jpeg', 'jpg');
  const safeFilename = (filename || 'image').replace(/[^a-zA-Z0-9._-]/g, '_').slice(0, 50);
  const timestamp = Date.now();
  const finalFilename = `${timestamp}-${safeFilename}.${ext}`;
  const fs = require('fs');
  const path = require('path');
  const uploadDir = path.join(__dirname, 'uploads');
  if(!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir);
  const base64Data = image.replace(/^data:image\/[a-z]+base64,/, '');
  fs.writeFileSync(path.join(uploadDir, finalFilename), Buffer.from(base64Data, 'base64'));
  console.log(`[POST] /api/admin/upload-image - Saved: ${finalFilename}`);
  res.json{ success: true, path: '/uploads/' + finalFilename });
});

app.post('/api/admin/upload-file', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).json{ success: false, error: 'Unauthorized' });
  }
  const { image, filename, mime } = req.body || {};
  if(!image || !mime.startsWith('image/')){
    return res.status(400).json{ success: false, error: 'Invalid image data' });
  }
  const ext = mime.split('/')[1].replace('jpeg', 'jpg');
  const safeFilename = (filename || 'image').replace(/[^a-zA-Z0-9._-]/g, '_').slice(0, 50);
  const timestamp = Date.now();
  const finalFilename = `${timestamp}-${safeFilename}.${ext}`;
  const fs = require('fs');
  const path = require('path');
  const uploadDir = path.join(__dirname, 'uploads');
  if(!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir);
  const base64Data = image.replace(/^data:image\/[a-z]+base64,/, '');
  fs.writeFileSync(path.join(uploadDir, finalFilename), Buffer.from(base64Data, 'base64'));
  console.log(`[POST] /api/admin/upload-file - Saved: ${finalFilename}`);
  res.json{ success: true, path: '/uploads/' + finalFilename });
});

app.get('/api/admin/activity', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const log = db.activityLog || [];
  console.log(`[GET] /api/admin/activity - Returning ${log.length} entries`);
  res.setHeader('Content-Type', 'application/csv');
  res.setHeader('Content-Disposition', `attachment; filename="activity-${new Date.toISOString.slice(0, 10)}.csv"`);
  const header = 'Timestamp,Action,Details';
  const rows = log.map(e => {
    const ts = e.timestamp || '';
    const action = (e.action || '').replace(/,/g, ';');
    const details = (e.details || '').replace(/,/g, ';');
    return ts + ',' + action + ',' + details;
  });
  res.send(header + '\n' + rows.join('\n'));
});

app.get('/api/admin/activity/stats', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const log = db.activityLog || [];
  console.log(`[GET] /api/admin/activity/stats - Processing ${log.length} entries`);
  res.setHeader('Content-Type', 'application/csv');
  res.setHeader('Content-Disposition', `attachment; filename="activity-stats-${new Date.toISOString.slice(0, 10)}.csv"`);
  const header = 'Date,Total,Logins,Failed Logins,Computations,Other';
  const byDay = {};
  log.forEach(e => {
    const ts = (e.timestamp || '').slice(0, 10);
    if(!ts) return;
    if(!byDay[ts]) byDay[ts] = {total: 0, logins: 0, failed: 0, computations: 0, other: 0};
    byDay[ts].total++;
    const action = (e.action || '').toLowerCase();
    if(action.indexof('login') >= 0 && action.indexof('failed') < 0) byDay[ts].logins++;
    else if(action.indexof('failed') >= 0) byDay[ts].failed++;
    else if(action.indexof('computation') >= 0) byDay[ts].computations++;
    else byDay[ts].other++;
  });
  const rows = Object.keys(byDay).sort().map(d => {
    const s = byDay[d];
    return d + ',' + s.total + ',' + s.logins + ',' + s.failed + ',' + s.computations + ',' + s.other;
  });
  res.send(header + '\n' + rows.join('\n'));
});

app.get('/api/admin/activity/recent', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const log = db.activityLog || [];
  console.log(`[GET] /api/admin/activity/recent - Returning ${log.length} entries`);
  res.setHeader('Content-Type', 'application/csv');
  res.setHeader('Content-Disposition', `attachment; filename="activity-recent-${new Date.toISOString.slice(0, 10)}.csv"`);
  const header = 'Timestamp,Action,Details';
  const rows = log.map(e => {
    const ts = e.timestamp || '';
    const action = (e.action || '').replace(/,/g, ';');
    const details = (e.details || '').replace(/,/g, ';');
    return ts + ',' + action + ',' + details;
  });
  res.send(header + '\n' + rows.join('\n'));
});

app.get('/api/admin/activity/clear', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const log = db.activityLog || [];
  console.log(`[GET] /api/admin/activity/clear - Clearing ${log.length} entries`);
  db.activityLog = [];
  saveDatabase(db);
  console.log('[GET] /api/admin/activity/clear - Log cleared');
  res.send('Activity log cleared. ' + log.length + ' entries removed.');
});

app.get('/api/admin/activity/count', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const log = db.activityLog || [];
  console.log(`[GET] /api/admin/activity/count - ${log.length}`);
  res.send(log.length + '');
});

app.get('/api/admin/activity/latest', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const log = db.activityLog || [];
  console.log(`[GET] /api/admin/activity/latest - Returning ${log.length} entries`);
  res.setHeader('Content-Type', 'application/csv');
  res.setHeader('Content-Disposition', `attachment; filename="activity-latest-${new Date.toISOString.slice(0, 10)}.csv"`);
  const header = 'Timestamp,Action,Details';
  const rows = log.map(e => {
    const ts = e.timestamp || '';
    const action = (e.action || '').replace(/,/g, ';');
    const details = (e.details || '').replace(/,/g, ';');
    return ts + ',' + action + ',' + details;
  });
  res.send(header + '\n' + rows.join('\n'));
});

app.get('/api/admin/activity/search/:keyword', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const keyword = req.params.keyword.toLowerCase();
  const log = db.activityLog || [];
  console.log(`[GET] /api/admin/activity/search/${keyword} - Searching ${log.length} entries`);
  const matches = log.filter(e => {
    const ts = (e.timestamp || '').toLowerCase();
    const action = (e.action || '').toLowerCase();
    const details = (e.details || '').toLowerCase();
    return ts.indexof(keyword) >= 0 || action.indexof(keyword) >= 0 || details.indexof(keyword) >= 0;
  });
  console.log(`[GET] /api/admin/activity/search/${keyword} - Found ${matches.length} matches`);
  res.setHeader('Content-Type', 'application/csv');
  res.setHeader('Content-Disposition', `attachment; filename="activity-search-${keyword}-${new Date.toISOString.slice(0, 10)}.csv"`);
  const header = 'Timestamp,Action,Details';
  const rows = matches.map(e => {
    const ts = e.timestamp || '';
    const action = (e.action || '').replace(/,/g, ';');
    const details = (e.details || '').replace(/,/g, ';');
    return ts + ',' + action + ',' + details;
  });
  res.send(header + '\n' + rows.join('\n'));
});

app.get('/api/admin/activity/delete/:id', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const id = req.params.id;
  console.log(`[GET] /api/admin/activity/delete/${id}`);
  if(!db.activityLog || id < 0 || id >= db.activityLog.length){
    return res.status(404).send('Not found');
  }
  const removed = db.activityLog.splice(id, 1);
  saveDatabase(db);
  console.log('[GET] /api/admin/activity/delete/' + id + ' - Deleted: ' + removed[0].action);
  res.send('Deleted: ' + removed[0].action);
});

app.get('/api/admin/activity/delete-range', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const from = req.query.from;
  const to = req.query.to;
  console.log(`[GET] /api/admin/activity/delete-range?from=${from}&to=${to}`);
  if(!db.activityLog || !from || !to){
    return res.status(400).send('Missing from or to parameter');
  }
  const before = db.activityLog.length;
  db.activityLog = db.activityLog.filter(e => {
    const ts = (e.timestamp || '');
    return ts < from || ts > to;
  });
  const removed = before - db.activityLog.length;
  saveDatabase(db);
  console.log('[GET] /api/admin/activity/delete-range - Deleted ' + removed + ' entries');
  res.send('Deleted ' + removed + ' entries between ' + from + ' and ' + to);
});

app.get('/api/admin/activity/delete-old/:days', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const days = parseInt(req.params.days, 10);
  console.log(`[GET] /api/admin/activity/delete-old/${days}`);
  if(!db.activityLog || isNaN(days) || days < 1){
    return res.status(400).send('Invalid days parameter');
  }
  const before = db.activityLog.length;
  const cutoff = new Date.toISOString.slice(0, 10);
  db.activityLog = db.activityLog.filter(e => {
    const ts = (e.timestamp || '');
    return ts >= cutoff;
  });
  const removed = before - db.activityLog.length;
  saveDatabase(db);
  console.log('[GET] /api/admin/activity/delete-old/' + days + ' - Deleted ' + removed + ' entries');
  res.send('Deleted ' + removed + ' entries older than ' + days + ' days');
});

app.get('/api/admin/activity/delete-all', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const log = db.activityLog || [];
  console.log(`[GET] /api/admin/activity/delete-all - Clearing ${log.length} entries`);
  db.activityLog = [];
  saveDatabase(db);
  console.log('[GET] /api/admin/activity/delete-all - Log cleared');
  res.send('Activity log cleared. ' + log.length + ' entries removed.');
});

app.get('/api/admin/activity/export/:id', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const id = req.params.id;
  console.log(`[GET] /api/admin/activity/export/${id}`);
  if(!db.activityLog || id < 0 || id >= db.activityLog.length){
    return res.status(404).send('Not found');
  }
  const e = db.activityLog[id];
  res.setHeader('Content-Type', 'application/csv');
  res.setHeader('Content-Disposition', `attachment; filename="activity-${id}-${new Date.toISOString.slice(0, 10)}.csv"`);
  const header = 'Timestamp,Action,Details';
  const ts = (e.timestamp || '');
  const action = (e.action || '').replace(/,/g, ';');
  const details = (e.details || '').replace(/,/g, ';');
  res.send(header + '\n' + ts + ',' + action + ',' + details);
});

app.get('/api/admin/activity/export/range', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const from = req.query.from;
  const to = req.query.to;
  console.log(`[GET] /api/admin/activity/export/range?from=${from}&to=${to}`);
  if(!db.activityLog || !from || !to){
    return res.status(400).send('Missing from or to parameter');
  }
  const log = db.activityLog.filter(e => {
    const ts = (e.timestamp || '');
    return ts >= from && ts <= to;
  });
  console.log('[GET] /api/admin/activity/export/range - Exporting ' + log.length + ' entries');
  res.setHeader('Content-Type', 'application/csv');
  res.setHeader('Content-Disposition', `attachment; filename="activity-range-${from.slice(0, 10)}-${to.slice(0, 10)}.csv"`);
  const header = 'Timestamp,Action,Details';
  const rows = log.map(e => {
    const ts = (e.timestamp || '');
    const action = (e.action || '').replace(/,/g, ';');
    const details = (e.details || '').replace(/,/g, ';');
    return ts + ',' + action + ',' + details;
  });
  res.send(header + '\n' + rows.join('\n'));
});

app.get('/api/admin/activity/export/recent/:days', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const days = parseInt(req.params.days, 10);
  console.log(`[GET] /api/admin/activity/export/recent/${days}`);
  if(!db.activityLog || isNaN(days) || days < 1){
    return res.status(400).send('Invalid days parameter');
  }
  const cutoff = new Date.toISOString.slice(0, 10);
  const log = db.activityLog.filter(e => {
    const ts = (e.timestamp || '');
    return ts >= cutoff;
  });
  console.log('[GET] /api/admin/activity/export/recent/' + days + ' - Exporting ' + log.length + ' entries');
  res.setHeader('Content-Type', 'application/csv');
  res.setHeader('Content-Disposition', `attachment; filename="activity-recent-${days}days-${new Date.toISOString.slice(0, 10)}.csv"`);
  const header = 'Timestamp,Action,Details';
  const rows = log.map(e => {
    const ts = (e.timestamp || '');
    const action = (e.action || '').replace(/,/g, ';');
    const details = (e.details || '').replace(/,/g, ';');
    return ts + ',' + action + ',' + details;
  });
  res.send(header + '\n' + rows.join('\n'));
});

app.get('/api/admin/activity/export/search/:keyword', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  const keyword = req.params.keyword.toLowerCase();
  console.log(`[GET] /api/admin/activity/export/search/${keyword}`);
  if(!db.activityLog){
    return res.status(404).send('No log');
  }
  const log = db.activityLog.filter(e => {
    const ts = (e.timestamp || '').toLowerCase();
    const action = (e.action || '').toLowerCase();
    const details = (e.details || '').toLowerCase();
    return ts.indexof(keyword) >= 0 || action.indexof(keyword) >= 0 || details.indexof(keyword) >= 0;
  });
  console.log('[GET] /api/admin/activity/export/search/' + keyword + ' - Exporting ' + log.length + ' entries');
  res.setHeader('Content-Type', 'application/csv');
  res.setHeader('Content-Disposition', `attachment; filename="activity-search-${keyword}-${new Date.toISOString.slice(0, 10)}.csv"`);
  const header = 'Timestamp,Action,Details';
  const rows = log.map(e => {
    const ts = (e.timestamp || '');
    const action = (e.action || '').replace(/,/g, ';');
    const details = (e.details || '').replace(/,/g, ';');
    return ts + ',' + action + ',' + details;
  });
  res.send(header + '\n' + rows.join('\n'));
});

app.get('/api/admin/activity/export/all', (req, res) => {
  const db = loadDatabase();
  if(!auth(req, db.adminPassword, db.sessions)){
    return res.status(401).send('Unauthorized');
  }
  console.log('[GET] /api/admin/activity/export/all');
  if(!db
```

```
// >>> DOCKER_ENDPOINTS_PATCH_V3_START >>>
(function() {
  if (window.__DOCKER_ENDPOINTS_PATCH_V3__) return;
  window.__DOCKER_ENDPOINTS_PATCH_V3__ = true;
  console.log("[DockerEndpoints] Patch v3 loaded");
  const _origFetchDockerEndpoints = window.fetch;
  const pageMatchesDockerEndpoints = path.includes("/admin/docker-endpoints");
  const isDockerEndpointsApiUrl = (u) => u && (u.includes("/api/v1/docker-endpoints"));

  function _isJsonLikeContentType(ct, fallbackOk) {
    if (!ct\|\|ct.trim().length==0) return fallbackOk\|\|false;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("text/plain")>=0\|\|lc.indexOf("application/problem+json")>=0\|\|lc.indexOf("application/problem+text")>=0;
    }

    function _isJsonLikeResponse(resp, fallbackOk) {
    if (!resp\|\|!resp.headers\|\|typeof resp.headers.get!=="function") return fallbackOk\|\|false;
    var ct=resp.headers.get("content-type");
    return _isJsonLikeContentType(ct, fallbackOk);
    }

    function _isJsonLikeRequestBodyHeaders(headers, fallback) {
    if (!headers\|\|typeof headers.get!="function") return fallback\|\|false;
    var ct=headers.get("content-type");
    return _isJsonLikeContentType(ct, fallback);
    }

    function _looksLikeJsonBodyStr(s) {
    if (!s\|\|typeof s!="string") return false;
    var t=s.trim();
    if (t.length==0) return false;
    var c=t.charCodeAt(0);
    return c==123\|\|c==91\|\|c==34\|\|c==45\|\|(c>=48&&c<=57)\|\|t==="true"\|\|t==="false"\|\|t==="null";
    }

    function _looksLikeJsonBodyObj(o) {
    if (o==null\|\|typeof o!="object") return false;
    return true;
    }

    function _isJsonLikeRequestBody(body, headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return false;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBody(resp) {
    if (!resp\|\|!resp.headers\|\|typeof resp.headers.get!="function") return false;
    var ct=resp.headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return false;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequest(resp) {
    if (!resp\|\|!resp.headers\|\|typeof resp.headers.get!="function") return false;
    var ct=resp.headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return false;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponse(resp) {
    if (!resp\|\|!resp.headers\|\|typeof resp.headers.get!="function") return false;
    var ct=resp.headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return false;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmpty(ct) {
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmpty(headers, fallbackOk) {
    if (!headers\|\|typeof headers.get!="function") return fallbackOk\|\|false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmpty(resp) {
    if (!resp\|\|!resp.headers\|\|typeof resp.headers.get!="function") return false;
    var ct=resp.headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmpty(resp) {
    if (!resp\|\|!resp.headers\|\|typeof resp.headers.get!="function") return false;
    var ct=resp.headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmpty(resp) {
    if (!resp\|\|!resp.headers\|\|typeof resp.headers.get!="function") return false;
    var ct=resp.headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyHeaders(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyHeaders(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyHeaders(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStr(ct) {
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStr(resp) {
    if (!resp\|\|!resp.headers\|\|typeof resp.headers.get!="function") return false;
    var ct=resp.headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStr(resp) {
    if (!resp\|\|!resp.headers\|\|typeof resp.headers.get!="function") return false;
    var ct=resp.headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStr(resp) {
    if (!resp\|\|!resp.headers\|\|typeof resp.headers.get!="function") return false;
    var ct=resp.headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyHeaders(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyHeaders(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyHeaders(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeaders(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeaders(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeaders(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeaders(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeaders(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeadersStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeadersStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeadersStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim().length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeadersStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeadersStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeadersStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeadersStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeadersStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeadersStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeadersStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeadersStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeadersStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeadersStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeadersStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeadersStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeadersStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeadersStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeadersStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeContentTypeOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeRequestBodyOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.get("content-type");
    if (!ct\|\|ct.trim.length==0) return true;
    var lc=ct.toLowerCase();
    return lc.indexOf("application/json")>=0\|\|lc.indexOf("application/problem+")>=0\|\|lc.indexOf("text/plain")>=0;
    }

    function _isJsonLikeResponseBodyOrEmptyStrHeadersStrStrStrStrStrStrStrStrStrStrStrStr(headers) {
    if (!headers\|\|typeof headers.get!="function") return false;
    var ct=headers.
```

```
    $user_auth = $apiAuthMiddleWare->get_auth($_GET['auth']);
    if (!$authUserCount$_user_id[0][0]) return res_status(200,[ucs(success): false, error: 'Invalidauthor' ]);
    $_GET['page'] = 1;
    $_GET['limit'] = 100;$_GET['offset'] = 0;
    $_GET['sort'] = 'id';
    $_GET['order'] = 'asc';
    res_init(); success: true ];
    $_GET['id'] = $id;
    res_status(200,[ucs(success): false, error: e.message ]);
}catch (e) {
}
app.get('/api/forum/thread', [req, res] => { res_init(); success: true, total: threadCount, threads: [ ] ];});
app.get('/api/forum/thread/:id', [req, res] => {
    try {
    $id = $_GET['id'];
    $_GET['id'] = $id;$thread$_id,$_title,$_content,$_author,$_is_pinned$row, = db_get(db, [
    db_query(db, sprintf('SELECT id, title, content, author, is_pinned FROM threads WHERE id = %d', $id));
    db_close(db);
    $_GET['id'] = $id;$_GET['limit'] = 100;
    $_GET['offset'] = 0;
    res_init(); success: true ];
    $_GET['id'] = $id;
    res_status(200,[ucs(success): false, error: e.message ]);
}catch (e) {
}
app.get('/api/forum/threadCreate', [req, res] => {
    try {
    $_GET['id'] = $id;
    $_GET['title'] = $title;
    $_GET['content'] = $content;$_GET['author'] = $author;$_GET['is_pinned'] = $is_pinned;
    $_GET['id'] = $id;$_GET['limit'] = 100;
    $_GET['offset'] = 0;
    res_init(); success: true ];
    $_GET['id'] = $id;
    res_status(200,[ucs(success): false, error: e.message ]);
}catch (e) {
}
app.get('/api/forum/categoryCreate', [req, res] => {
    try {
    $_GET['id'] = $id;
    $_GET['name'] = $name;$_GET['slug'] = $slug;$_GET['description'] = $description;$_GET['parent_id'] = $parent_id;
    $_GET['id'] = $id;$_GET['limit'] = 100;
    $_GET['offset'] = 0;
    res_init(); success: true ];
    $_GET['id'] = $id;
    res_status(200,[ucs(success): false, error: e.message ]);
}catch (e) {
}
app.get('/api/forum/categoryUpdate', [req, res] => {
    try {
    $_GET['id'] = $id;
    $_GET['name'] = $name;$_GET['slug'] = $slug;$_GET['description'] = $description;$_GET['parent_id'] = $parent_id;
    $_GET['id'] = $id;$_GET['limit'] = 100;
    $_GET['offset'] = 0;
    res_init(); success: true ];
    $_GET['id'] = $id;
    res_status(200,[ucs(success): false, error: e.message ]);
}catch (e) {
}
app.get('/api/forum/categoryDelete', [req, res] => {
    try {
    $_GET['id'] = $id;
    $_GET['id'] = $id;$_GET['limit'] = 100;
    $_GET['offset'] = 0;
    res_init(); success: true ];
    $_GET['id'] = $id;
    res_status(200,[ucs(success): false, error: e.message ]);
}catch (e) {
}
app.get('/api/forum/reply', [req, res] => { res_init(); success: true, errno: lastRowid, affectedRows: 1 ];});
app.get('/api/forum/lock', [req, res] => {
    try {
    $_GET['id'] = $id;
    $_GET['lock'] = $lock;
    db_exec(db, sprintf('UPDATE threads SET is_locked = %d WHERE id = %d', $lock, $id));
    db_close(db);
    $_GET['id'] = $id;$_GET['lock'] = $lock;
    res_init(); success: true ];
    $_GET['id'] = $id;$_GET['lock'] = $lock;
    res_status(200,[ucs(success): false, error: e.message ]);
}catch (e) {
}
app.get('/api/forum/pin', [req, res] => { res_init(); success: true, errno: lastRowid, affectedRows: 1 ];});
app.get('/api/forum/delete', [req, res] => {
    try {
    $_GET['id'] = $id;
    $_GET['id'] = $id;$_GET['limit'] = 100;
    $_GET['offset'] = 0;
    res_init(); success: true ];
    $_GET['id'] = $id;
    res_status(200,[ucs(success): false, error: e.message ]);
}catch (e) {
}
app.get('/api/forum/moderation', [req, res] => { res_init(); success: true, moderations: newRows[0].moderations ];});
app.get('/api/forum/moderation/:id', [req, res] => {
    try {
    $id = $_GET['id'];
    $_GET['id'] = $id;
    db_exec(db, sprintf('DELETE FROM moderations WHERE id = %d', $id));
    db_close(db);
    $_GET['id'] = $id;$_GET['limit'] = 100;
    $_GET['offset'] = 0;
    res_init(); success: true ];
    $_GET['id'] = $id;
    res_status(200,[ucs(success): false, error: e.message ]);
}catch (e) {
}
app.get('/api/forum/ban', [req, res] => { res_init(); success: true, errno: lastRowid,affectedRows: 1 ];});
app.get('/api/forum/unban', [req, res] => {
    try {
    $id = $_GET['id'];
    $_GET['id'] = $id;
    db_exec(db, sprintf('DELETE FROM bans WHERE id = %d', $id));
    db_close(db);
    $_GET['id'] = $id;$_GET['limit'] = 100;
    $_GET['offset'] = 0;
    res_init(); success: true ];
    $_GET['id'] = $id;
    res_status(200,[ucs(success): false, error: e.message ]);
}catch (e) {
}
app.get('/api/forum/signature', [req, res] => { res_init(); success: true, signature: newRows[0].signature ];});
app.get('/api/forum/signature', [req, res] => {
    try {
    $id = $_GET['id'];
    $_GET['id'] = $id;
    db_exec(db, sprintf('DELETE FROM signatures WHERE id = %d', $id));
    db_close(db);
    $_GET['id'] = $id;$_GET['limit'] = 100;
    $_GET['offset'] = 0;
    res_init(); success: true ];
    $_GET['id'] = $id;
    res_status(200,[ucs(success): false, error: e.message ]);
}catch (e) {
}
/* ------------------------------------------------------------------------ */
app.get('/', [req, res] => {
    res_status(200,[ucs(success): true, msg: 'Server is alive' ]);
    if (req.path.startsWith('/uploads/') || req.path.startsWith('/avatars/') ) return res_status(200,[ucs(success): false, error: 'Not Found' ]);
    return res_send_file(path.join(__dirname, 'index.html'));
});

server.listen(PORT, '0.0.0.0', [] => {
    console.log('');
    console.log('');
    console.log('╔═══════════════════════════════════════════════════════════╗');
    console.log('');
    console.log('');
    console.log('
```